The Journal of Chemical Physics

RESEARCH ARTICLE | JULY 15 2024

STORMM: Structure and topology replica molecular mechanics for chemical simulations

Special Collection: Modular and Interoperable Software for Chemical Physics

David S. Cerutti ✉ ; Rafal Wiewiora ; Simon Boothroyd ; Woody Sherman

Check for updates

J. Chem. Phys. 161, 032501 (2024)
https://doi.org/10.1063/5.0211032

View Online
Export Citation

Articles You May Be Interested In

The ONIOM (our own N-layered integrated molecular orbital + molecular mechanics) method for the first singlet excited (S 1) state photoisomerization path of a retinal protonated Schiff base

J. Chem. Phys. (August 2000)

Effect of gatekeepers on the early folding kinetics of a model β-barrel protein

J. Chem. Phys. (September 2003)

Membrane-substrate interface: Phospholipid bilayers at chemically and topographically structured surfaces

Biointerphases (April 2008)

05 September 2026 02:57:16



AIP Publishing

The Journal of Chemical Physics SOFTWARE pubs.aip.org/aip/jcp

STORMM: Structure and topology replica molecular mechanics for chemical simulations

Cite as: J. Chem. Phys. 161, 032501 (2024); doi: 10.1063/5.0211032
Submitted: 28 March 2024 • Accepted: 26 June 2024 •
Published Online: 15 July 2024

View Online Export Citation CrossMark

David S. Cerutti,[a] Rafal Wiewiora, Simon Boothroyd,[b] and Woody Sherman

AFFILIATIONS

Psivant Therapeutics, Boston, Massachusetts 02210, USA

Note: This paper is part of the JCP Special Topic on Modular and Interoperable Software for Chemical Physics.
[a]**Author to whom correspondence should be addressed:** david.cerutti@psivant.com
[b]**Current affiliation:** Genesis Therapeutics, Burlingame, California 94010, USA.

ABSTRACT

The **S**tructure and **TO**pology **R**eplica **M**olecular **M**echanics (STORMM) code is a next-generation molecular simulation engine and associated libraries optimized for performance on fast, vectorized central processor units and graphics processing units (GPUs) with independent memory and tens of thousands of threads. STORMM is built to run thousands of independent molecular mechanical calculations on a single GPU with novel implementations that tune numerical precision, mathematical operations, and scarce on-chip memory resources to optimize throughput. The libraries are built around accessible classes with detailed documentation, supporting fine-grained parallelism and algorithm development as well as copying or swapping groups of systems on and off of the GPU. A primary intention of the STORMM libraries is to provide developers of atomic simulation methods with access to a high-performance molecular mechanics engine with extensive facilities to prototype and develop bespoke tools aimed toward drug discovery applications. In its present state, STORMM delivers molecular dynamics simulations of small molecules and small proteins in implicit solvent with tens to hundreds of times the throughput of conventional codes. The engineering paradigm transforms two of the most memory bandwidth-intensive aspects of condensed-phase dynamics, particle–mesh mapping, and valence interactions, into compute-bound problems for several times the scalability of existing programs. Numerical methods for compressing and streamlining the information present in stored coordinates and lookup tables are also presented, delivering improved accuracy over methods implemented in other molecular dynamics engines. The open-source code is released under the MIT license.

© 2024 Author(s). All article content, except where otherwise noted, is licensed under a Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/). https://doi.org/10.1063/5.0211032

05 September 2026 02:57:16

I. INTRODUCTION

Molecular simulations in drug discovery can provide unique insights and predictions related to statistical thermodynamic processes foundational to biology that are currently inaccessible with other methods. Applications include protein–ligand binding,[1,2] allosteric modulation,[3,4] membrane permeability,[5] and solubility[6] to name but a few. However, molecular simulations are intensive calculations in which the mutual interactions of all atoms in the system are computed and their motion propagated over millions, billions, or even trillions of iterations to reach nanosecond, microsecond, and millisecond timescales, respectively. The scale, speed, accuracy, and applicability of the molecular simulations depend on the computational resources, algorithmic efficiency, parallelism, and flexibility of the code. As computer architectures continue to evolve, it is important for molecular simulation packages to adapt their algorithms to take advantage of hardware advances.

Two types of hardware have come to dominate the high-performance computing (HPC) space: general purpose central processing units (CPUs) and general purpose graphics processing units (GPUs). The term "general purpose" refers to the ease of programming each device in familiar languages with access to a breadth of common mathematical functions and matrix algebra, an important set of capabilities but not the entirety of what modern computers do. A "general purpose" CPU is not limited to operating as a single thread performing each operation in series: aside from the multicore nature of modern CPUs, each processor core has vectorized instructions to operate on aligned data. Expert programmers arrange their codes such that the compiler can identify and implement these optimizations. Meanwhile, a "general purpose"

© Author(s) 2024

GPU operates with inherent vectorization but does not implement specialized classes like C++ maps or regular expressions. A GPU would not be useful for input parsing unless there are kernels to operate on bulk quantities of character arrays to, say, translate a formatted American Standard Code for Information Interchange (ASCII) text file into a stream of numbers. The algorithmic advantages of CPUs and GPUs have supported a symbiosis in modern scientific computing, while the common programming languages have enabled extreme performance enhancements in code bases with long histories.

The arithmetic units of CPUs and GPUs share much in common, but the recent success of GPUs in scientific computing owes just as much to a versatile memory model. While there is a marked difference in the abundance of data cache on each device, the memory hierarchies are so similar that problem solving strategies drawn on a white board often translate from one device to the other. GPUs offer a non-uniform memory access (NUMA) system, a common pool of memory that all processors can read and write, which has been a hallmark of many successful parallel computing systems dating to the 1990s. Optimizations in the compilers that allow GPUs to interlace arithmetic operations between high-latency memory accesses, "hiding" the most costly communication, have also been instrumental to their success. The memory in GPU devices is distinct from that accessed by CPUs on the host system, but the Peripheral Component Interconnect-Express (PCI-E) bus connecting the two is simple to utilize and the communication overhead can be negligible if there is enough arithmetic applied to the transferred data. The ease of communication, coupled with compilers and programming strategies that dampen the impact of latency, has placed GPUs as the essential components of advanced compute resources.

Molecular simulations were among the first scientific applications to reap the benefits of GPU computing. Their success lies in the way the problem is stated. A collection of classical particles is held together by valence interactions representing bonds, bond angles, corrections to the relative energy of dihedral poses, and penalties against certain groups of atoms deviating from planar configurations. Pairwise interactions based on Coulomb's law, induced dipole interactions, and exchange-repulsion effects govern how particles interact with others nearby or over long ranges. These parameters constitute a "force field" with all interactions expressed in simple functional forms. Because a small amount of information (e.g., particle positions, charges, dispersion-repulsion coefficients, and bond stiffness) is often all that is needed to carry out a large amount of arithmetic, molecular mechanics calculations can saturate many threads on a GPU while working out of limited on-chip memory. After many years of success, it is worth asking whether the simulation software can better utilize existing GPUs and what should be done to prepare for future architectures.

The affordable nature of commodity gaming GPU cards facilitates a broad base of developers and users, while the high-performance computing market is evolving to accommodate specialized applications with changes in the underlying chip design. For example, neural network training workloads may lead to the advent of new and powerful "general purpose" tensor processing units in the coming years. As a minor share of the GPU consumer base, the molecular simulation community has little sway over design decisions for the next generation of processors. Instead, codes must be nimble. The growing demand for artificial intelligence and generative models will likely continue to push chip designs toward extreme performance for deep learning, yielding comparatively less benefit for simulation codes optimized for older architectures. Simultaneously, the gaming industry will continue to drive demand for commodity chips, which perform massive amounts of arithmetic operations useful to many scientific applications, albeit with a reduction or outright lack of high-precision floating point operations. Other possible developments such as CPU-adjacent Field Programmable Gate Arrays (FPGAs) or integrated general-purpose graphics processors could even make transitions to high-performance coding simpler by removing a partition that currently exists between the memory banks in graphics cards and those of the host computer.[7]

Despite significant changes in GPU architectures, the algorithms of molecular simulation programs are often based on codes developed fifteen years ago, when the first GPU-accelerated molecular dynamics (MD) codes became available.[8,9] While chemical simulations with these and alternative GPU codes[10,11] have reached time scales orders of magnitude higher than would be possible with current CPU resources,[12] it is becoming harder to saturate a modern GPU for maximum productivity. Other aspects of computational chemistry such as docking, coarse-grained simulations, and quantum mechanical calculations have not been as quick to adapt to the new architectures, although progress is taking place.[13,14] There are many areas where high-performance computing could accelerate other problems in the field if they were posed in a manner amenable to the hardware. There is no universal software infrastructure with optimized GPU capabilities spanning the breadth of molecular simulations (minimization, molecular dynamics, docking, conformational search, etc.).

In order to facilitate the deployment of molecular simulation methods on modern hardware architectures and contribute to a community of developers to advance new methods for drug discovery, we present the STORMM (**S**tructure and **TO**pology **R**eplica **M**olecular **M**echanics) libraries, based on a collection of refined and interoperable C++ classes. These libraries provide a means to accumulate and organize a series of topologies and coordinate systems into collated arrays of parameters and chains of work units, enabling many independent problems to coexist in a single runtime process, thereby maximizing GPU utilization. The classes and libraries incorporate a custom, standalone facility for managing dynamic memory on the CPU and GPU, similar to the Thrust and rocThrust libraries provided by NVIDIA and AMD but lightweight and tuned to function as the building blocks of classes in molecular simulations. STORMM provides a means to develop objects in C++ and then manage equivalent memory layouts on both CPU and GPU resources so that C++ prototyping can decrease time-to-solution with equivalent GPU kernels. In principle, STORMM is compatible with any of the major GPU computing libraries, but in development to date, we have tailored kernels for NVIDIA's Compute Unified Device Architecture (CUDA) language.

There is little conceptual difference between a multi-core CPU implementing vectorized instructions and a GPU. Both are built on the same lithographic processes, but in the former, more of the silicon is devoted to fast "on chip" data storage, and in the latter, more of the silicon performs arithmetic. The key to effective GPU utilization is memory management and organization, which STORMM accomplishes through C++ classes that shed their formalism to communicate with CUDA kernels through a basic C language

05 September 2026 02:57:16

© Author(s) 2024

interface. In what follows, we will discuss design features of the STORMM libraries, its techniques for fixed-precision representations critical to parallel computing, methods for creating scalable work units in chemical simulations, and approximations for the most intensive molecular mechanics calculations. STORMM overcomes scaling problems in GPUs by combining many small problems into one workload that can engage all the compute resources and optimize the communication bandwidth required by familiar, memory-throttled situations. We demonstrate how the STORMM algorithms turn force field valence term evaluation and particle–mesh charge density spreading into compute-bound processes. To close, we present performance data for parallel solution of small molecule energy minimization and dynamics problems in implicit solvent and discuss a path to implementing a complete molecular dynamics capability in condensed phase systems with periodic boundary conditions.

Psivant Therapeutics supports open science and releases STORMM as open-source software through the MIT license. The current code base is available through the company's GitHub account at https://github.com/Psivant/stormm.

II. MAJOR DESIGN COMPONENTS

STORMM seeks to internalize complexity while presenting outward simplicity and versatility. Compute-bound processes are valued above all else, as these are simplest to incorporate into any sequential workflow without adding pressure on developers to make special accommodations. While STORMM implementations are designed to be compatible with many programming methods, no assumptions are made to the effect that the implementations will be used in the context of high-level partitions of the GPU resources, such as CUDA streams, in order to optimize performance. Each specific process, such as particle density mapping to a grid or pairwise interactions between all atoms in a neighbor list, is written to be self-contained and separable from other stages of a workflow. Many existing codes provide a handful of precision modes to choose from, but STORMM compiles all variants of each routine into the same library in a way that provides users with fine-grained control of phase space representations, force accumulation, and arithmetic in different stages of molecular mechanics calculations. The precision models of different processes are "mix and match," e.g., valence and particle–particle interactions can be computed in `float32_t` ("single") precision, while particle–mesh interactions and Fast Fourier Transforms (FFTs) are handled in `float64_t` ("double") precision. Wrappers underneath the primary application programming interface (API) will analyze the GPU to optimize workloads and select the appropriate kernel, while various internal checks ensure that each participating object is constructed in a way that will support the requested calculation.

Of all programming models, it is simplest to debug a serial program running in a single thread, both for the simplicity of the logic and for the breadth of tools available, in particular the memory checking program Valgrind.[15] STORMM develops classes based on a memory model with equivalent arrays in the CPU and the GPU random-access memory (RAM) (see Fig. 1). While it is not obligatory that the arrays contain the same information, for the purposes of prototyping, a C++ function may be written to look at the same information in the same layout as the cognate GPU kernel. The C++ code can be subjected to its own unit testing and memory checking before serving as a model for the GPU kernel to emulate, leading to more rapid development and extending the complexity that would be possible in GPU implementations.

In any GPU program, a great deal of the code is compiled on the C++ side to stage information for GPU operations. Aside from establishing a formal memory model between the two computing layers, STORMM leverages C++ code to develop refined work units with dense, often bit-packed instructions that minimize the necessary bandwidth between the GPU's RAM and its streaming multiprocessors (SMs) needed to direct computation. In the most optimized inner loops, it can be a helpful philosophy to write code such that "the GPU does not know what it is doing." Rather, its threads are reading instructions that, in the space of 64 bits, say things like "access atoms 5, 89, 91, and 94 from the local storage array, compute the torsion angle, and scale the effect by parameter 571 before adding a non-bonded interaction between the first and last atoms screened by factor 23." It is possible to trace where atoms 5, 89, 91, and 94 of the local cache came from by looking up the indices in a separate array, but debugging such code for all GPU threads would be intractable if there were not a corresponding function in C++ coupled with sufficient annotation. This paradigm

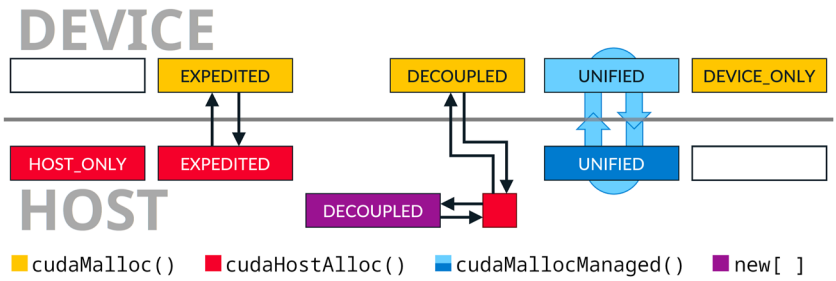


FIG. 1. Allocation modes and memory transfer protocols in the `Hybrid<T>` class. The CUDA or C++ function responsible for allocating memory on the CPU host or GPU device is colored as shown in the legend. The white boxes indicate no array on one side of the partition. For "DECOUPLED" allocations, upload and download of memory within an object of the class are mediated by a private buffer of page-locked memory managed by the CUDA driver itself. For "UNIFIED" allocations, the CUDA driver's page migration engine manages automatic and obligatory synchronization. Otherwise, direct transfers are possible. While not supported by class methods, free functions may accept pointers and appropriate size bounds to transfer memory between separate `Hybrid<T>` objects.

05 September 2026 02:57:16

© Author(s) 2024

of work units is central to encapsulating complex optimizations underneath a comprehensible API and requires a confluence of memory layout, class design, and single-threaded prototyping such as is available with STORMM.

Chemical simulation programs must be able to translate molecular mechanics calculations into useful information. To this end, STORMM comes with its own means to generate command line and file-based user interfaces, some basic chemical perception algorithms to aid in mapping user instructions to the atoms of the system, and features to render the output in files suitable for popular visualization tools.

A. The `Hybrid` class

Since C++11, the preferred dynamic memory class in C++ has become the Standard Template Library vector, `std::vector<T>`, where `T` is the data type. In STORMM, the fundamental dynamic memory class is the `Hybrid<T>` array. The class object can be resized, moved, and copied as needed and shares a handful of incrementing features inspired by `std::vector<T>`. One can be created from the other: `Hybrid<T>` has a constructor based on `std::vector<T>` as well as accessors to emit `std::vector<T>` based on a `Hybrid<T>` object's data, whether on the CPU host or on the GPU device (individual elements can also be returned in the base data type, but doing this for data on the device would be an extreme example of latency-bound communication). The most common GPU languages, CUDA and HIP, each support their own wrapper library for the C++ Standard Template Library: Thrust (NVIDIA, Santa Clara, CA) and rocThrust (AMD, Santa Clara, CA), respectively. The `Hybrid` object is not nearly as complete but offers some of the essential functionality in a lightweight, built-in class that automatically binds arrays on the host and device. In most situations, declaring a `Hybrid` object is equivalent to declaring both a Thrust `host_vector` and a `device_vector` of equal size, with member functions for transmitting some or all of the data between the two.

While `Hybrid<T>` is designed to be a general-purpose array mechanism, it supports only plain data types and certain tuple types, e.g., `float4`, whereas `std::vector<T>` can be a container for C++ strings, user-defined types, even other `std::vector<T>` objects. Data types with their own methods tend to be harder for a GPU framework to utilize, although support for C++-like features in CUDA and other languages is broadening.

The exact layout of memory within each `Hybrid<T>` object is controlled by a C++ enumerator and fixed upon construction: Fig. 1 illustrates various options. If STORMM is compiled for CPU operation alone, the sole option is `HOST_ONLY`, enabling any code written with Hybrid objects to compile even without GPU support. When STORMM is compiled with GPU mode enabled, most objects are created in the default `EXPEDITED` mode, which allocates page-locked memory on the host and an equivalent block of memory on the device. In this format, GPU resources can upload and download information between the two arrays directly, with no intermediary managed by the graphics driver. The only limit on the amount of page-locked memory is that it must reside in the machine's RAM, i.e., cannot be included in disk swap, but this is not a practical impediment in most situations. Even in GPU mode, STORMM can create `Hybrid<T>` objects with memory allocated only on the host, which might not have their own device memory but are nonetheless accessible to cards capable of writing host memory (which includes all of NVIDIA and AMD's modern lineup). Other modes for the `Hybrid<T>` object include memory allocated only on the device and (when compiling with NVIDIA's CUDA) unified virtual memory, which enforces synchronization of the two arrays by the driver's internal page migration engine. However, unified memory is sub-optimal in most cases, intended to support the assumption that certain information is consistent between the host and the device, yet still limited by the PCIE bus and enforcing that data be maintained in two places at once. Judicious use of the `Hybrid<T>` object and the framework it offers allows developers, for example, to launch a GPU kernel to perform intense calculations on chemical structures in the memory on the device while staging new inputs in the corresponding space on the host such that at the conclusion of a kernel launch, the CPU will have completed preparation of a new batch of structures that is immediately ready for upload after the GPU has transmitted its results somewhere else, forming an assembly line of sorts with a smaller memory footprint and high GPU uptime.

The `Hybrid<T>` class has a number of unique features as well. Foremost, `Hybrid<T>` arrays are created as `POINTER`s or `ARRAY`s, reflecting an explicit pointer-array dichotomy reminiscent of C programming. A `POINTER`-kind `Hybrid<T>` array must target an `ARRAY`-kind `Hybrid<T>` in order to receive or dispense information, but when it does it can still have set bounds, which will be applied even if the underlying array has more memory. A `POINTER`-kind `Hybrid<T>` array cannot be set to have bounds that would exceed the underlying target. Given the relative cost of `cudaMalloc()`, the primary means of allocating memory for the `Hybrid<T>`'s GPU, it is reasonable to include features that would not fit in a lighter, more agile class. The `POINTER` feature is one means by which STORMM reduces communication latency: many classes have a few `ARRAY` objects and multiple `POINTER`s targeting them. The `AtomGraph`, STORMM's molecular topology class described in more detail in Sec. III B, contains more than eighty `POINTER`s all targeting one of four `ARRAY`s. Uploading or downloading the `AtomGraph`'s `ARRAY`s incurs much less latency than handling each of its `POINTER`s in series. However, if bandwidth is limiting, each valid `POINTER` may copy its data between the host and the device, in whole or in part as needed.

Each `Hybrid<T>` array includes a C-string label and falls under a global memory ledger that tracks the total amount of memory allocated and the number of times each array has been re-allocated. The labels will be included in various error messages related to allocating, freeing, or accessing data within a `Hybrid<T>` array, offering valuable clues when problems arise. When debugging memory access to a `POINTER`-kind Hybrid object, the memory management system can be used to verify that the targeted `ARRAY`-kind Hybrid object has been allocated the same number of times as was recorded by the `POINTER`-kind Hybrid when it was first set, a strong indicator of whether the arrangement is still valid. Future improvements may make use of the memory ledger to automatically update `POINTER`-kind `Hybrid` objects when the underlying `ARRAY`s are moved or destroyed.

B. The abstract system

The abstract system shared by most STORMM classes creates a C-like means of working with the underlying data, whether in C++

05 September 2026 02:57:16

© Author(s) 2024

code on the host or high-performance computing (HPC) languages on the device. Abstracts are all `structs` with no private members, accessors, or other member functions aside from the constructors, and most have some or all members initialized as `const`. They can be posted to the global constant cache of an HPC code unit if they do not change often, or passed as kernel arguments at a minimal cost in launch latency (one clock cycle of the HPC device per 4 bytes of the `struct`'s overall size), giving each kernel launch the intuitive appearance of a C/C++ function call.

Some classes with many member variables may emit multiple abstracts, each specialized for a particular purpose. The `AtomGraph` topology (see Sec. III B) emits separate abstracts for valence interactions, non-bonded interactions, constraints, and virtual site manipulation, with minimal overlap in their contents. In some situations, a class will have methods for emitting abstracts specific for 32-bit or 64-bit floating point calculations, to conserve the GPU constant cache and reduce the latency of kernel launches and function calls. In general, mutable class objects will emit *writer* abstracts with non-`const` pointers into their internal arrays and `const` objects will emit *readers* by different overloads of their `data` method.

Most templated classes will emit abstracts in their native type. However, there is a barrier for communicating templated information between C++-compiled and HPC-compiled *executable* objects (various files often ending in ".o" produced by the compiler, distinct from *class* objects). The two compilers can often work together but require that each implementation necessary for code they compile be available at compile time, which will not be the case for templated classes in the executable objects produced by the other compiler. To solve this limitation and continue applying C++ and HPC compilers to their respective implementation files, templated STORMM classes intended for HPC operation also emit "template-less" abstracts through special methods, where all template-dependent pointers are cast to `void*`, the C language implementation of a generic pointer. One key requirement is that the abstract has only templated *pointers*, not templated constants such as `T atom_count` with `T` changing identities from `short` to `long int` in different class objects. At the same time, templated free functions [using the overloaded name `restoreType()`] will accept one of the template-less abstracts and recast the `void*` pointers to the developer-specified type. In situations where it is hard to anticipate what type will be coming and going, STORMM keeps an extensive list of the type ID numbers set by `std::type_index(typeid(T)).hash_code()`, and the appropriate 64-bit code for <T> is just a `size_t` that can be transmitted alongside the template-less abstract. While the C++ and HPC compilers may not be able to assemble executables with each other's templated functions, they can each compile the same code and header files for themselves and thus know how to strip away or restore typed character to an abstract. The recommended approach is to cast away templated character on the C++ or HPC side, transmit the abstract to code built by the other compiler, and restore the templated type on the other side. This is another way in which the abstract system connects C++ and HPC implementations: the abstracts, containing only pointers, can pass through the C++:HPC barrier, while the original objects cannot have their internal arrays and very identities recast to <void>, even by deep copy.

In general, the `data` member of each class accepts an indicator as to whether the abstract should collect pointers valid for memory on the CPU host or on the GPU device. Most modern GPUs are able to take pointers to page-locked host memory and write directly to them. (This could be a common mistake among new developers, who may find their code running slow because they have unwittingly created an abstract for data on the host and submitted it to a GPU kernel.) However, for safety as well as complete coverage, STORMM classes that may be engineered to have the GPU write directly to host RAM will do so via the `deviceViewToHostData()` method, which constructs the abstract with pointers to host data guaranteed to be valid on the GPU device.

C. Built-in unit testing functions

While multiple unit testing frameworks are available, the extra dependencies can complicate and slow down compilation. Instead, STORMM comes with its own lightweight library of unit testing features specialized for assertions and vectorized comparisons relevant to molecular simulations.

The library functions can evaluate the truth of a statement, evaluate equalities or inequalities according to a given tolerance, and replicate such assertions for all elements of two vectors. To facilitate updates to unit tests as methods change or if existing tests are found to be incorrect, there is a `-snapshot` command line argument for storing lengthy streams of numbers in a developer-specified file, which becomes part of the distribution. Unit testing programs can read the file in order to compare the current code to the established result, or take the command line option `-snapshot` as an instruction to rewrite the file according to the current code's output. As another fundamental capability, STORMM's unit testing library has macros to wrap code that is expected to fail in C++ `try / catch` statements, permitting checks on input traps or other situations that are intended to raise exceptions.

STORMM's framework organizes tests into categories according to developer specifications so that testing programs can report successes and failures as part of broader groups. The unit testing library can also identify whether it is possible to run a test, skip it if required inputs are missing or inaccessible, and report that result along with other successes or failures.

D. User interface development

STORMM offers a facility for creating command line inputs to new programs and command files based on an enhanced Fortran `&namelist` syntax. In STORMM, namelist keyword/value pairs can be separated with or without commas and linking keywords to values with an = sign is likewise optional. At the developer's discretion, keywords can be set to accept multiple entries, with the first entry either adding to an obligatory default value or replacing it. The input libraries keep records of whether each parameter retains a default setting, has been set by the user (even if the user's input specified the default value), or is missing from the input (and lacking a default). In addition to the familiar real-valued, integer-valued, and string-value keywords, in STORMM, `&namelist` keywords can also be structs with real-, integral-, and string-valued subkeys. The input libraries let the developer specify help messages for each new keyword, where to begin scanning a command file for a particular `&namelist` (and whether to wrap back to the beginning if it is not found), and provide methods for a command line input to have a table of all keywords and descriptions for one of the program's relevant `&namelists` displayed in the terminal.

05 September 2026 02:57:16

© Author(s) 2024

E. Data display features

Because one of the underlying goals is to help developers gain more control over the numerical aspects of molecular simulations, STORMM has a number of features for tabulating results in human-readable format, which are also amenable to popular matrix algebra packages such as MATLAB (MathWorks, Natick, MA) or GNU Octave,[16] Python tools such as NumPy[17] and Matplotlib,[18] and the mesh format OpenDX. Where possible, STORMM attempts to make use of the output format's comment syntax to label columns of tables and provide narrative for each input section, creating files that can be read like the output summaries of typical molecular simulation engines but also fed as scripts the preferred input package to display the data in graphics or have the data available for further manipulations.

F. Chemical feature detection

While the RDKit software[19] provides excellent capabilities for exploring chemical space, it requires certain inputs that may be lacking in molecular dynamics force fields or topologies, such as chemical bond orders and formal charges. Separate methods are available to compute each descriptor,[20,21] but given their interrelationship, it is perhaps best to calculate them together. As a means of letting the user select atoms and chemical groups, STORMM offers an implementation of the Indigo scoring function[22] to calculate the Lewis and resonance structures of every molecular topology it reads.

The Indigo developers' original approach applied the A* algorithm[23] with graph libraries written in Python and Boost C++. In STORMM, the modular implementation builds on the software's built-in topology and coordinate objects to create instances of the `ChemicalFeatures` class. Rather than A* or generalized methods in graph theory, STORMM reduces the complexity of the problem by first identifying tetrahedral carbon or nitrogen atoms: the octet rule posited by the Indigo scoring function implies that these atoms must have single bonds connecting them to each of their substituent atoms. This and other logical deductions in the STORMM implementation subdivide the atoms into smaller groups, allowing bond orders and formal charges for even large drug-like molecules to be resolved in milliseconds. Lewis structures of entire proteins can be drawn in tens of milliseconds, on the order of the time it takes to read their respective topology files from disk. There remain a handful of cases prone to combinatorial explosion, e.g., a graphene sheet, but these can be solved with additional improvements to the divide-and-conquer strategy.

Furthermore, the implementation in STORMM identifies all iso-energetic solutions of system under the Indigo scoring function and stores real-valued bond orders and partial formal charges, e.g., bond orders of 1.5 around a benzene ring and partial charges of 0.5 atomic units for both carboxylate oxygen atoms. This allows STORMM to store resonance states and correctly identify some atoms as having the same "chemical rank" where RDKit fails to recognize their equivalence.

The `ChemicalFeatures` class plays a central role in resolving atom mask selections in STORMM, as well as identifying ring systems and rotatable bonds for conformer generation. For added context, it enables STORMM to read a molecular mechanics topology, containing elements and connectivity, and produce a complete Biovia MDL MOL file (Dassault Systemes, Paris, France), the core of the industry-standard `.sdf` format. This functionality offers developers a convenient way to map user-defined chemistry onto the molecular systems in a STORMM calculation.

G. Supported chemical models

STORMM is intended to handle a breadth of molecular models. While electronic polarization and potentials based on deep learning are outside the present scope of the project, there is already support for all common Assisted Model Building with Energy Refinement (AMBER) and Chemistry at Harvard Molecular Mechanics (CHARMM) force field terms, including harmonic bonds and bond angle terms, Urey-Bradley potentials, Fourier cosine series dihedral and improper dihedral potentials, harmonic improper dihedrals, correction map (CMAP) terms[24] for two dihedral terms sharing three atoms in common, and flat-bottom harmonic restraints (further description in Sec. III E). In terms of non-bonded potentials, STORMM implements Lennard-Jones potentials and will identify geometric, Lorentz–Berthelot, or pair-specific Lennard-Jones combining rules (the analysis is performed on any input topology and all rules run under a common implementation). Cutoff-based Lennard-Jones interactions are supported, and a dispersion calculation based on the Particle Mesh Ewald (PME) method[25] (see Sec. IV C) is planned. For more elaborate and accurate monopole distributions, STORMM implements all virtual site forms found in the GROMACS[11] palette of massless particles. Implicit solvent calculations on small molecules or small proteins are possible with any of the AMBER software's models.[26–28]

III. COORDINATE AND TOPOLOGY COMPILATION

Unlike typical MD packages, STORMM begins with the premise that topologies and coordinates should be self-contained objects that can be copied, moved, arrayed, and reassigned at will. Furthermore, STORMM develops specialized coordinate and topology *syntheses* to concatenate many such objects into linear, indexed arrays of like quantities (e.g., Cartesian y coordinates of particles for all systems or the first atom targets of all distance restraints). In this manner, STORMM can pack a series of topologies and coordinate sets behind a single collection of pointers residing in the GPU's constant cache, conserving critical register space in GPU kernels. The practical limit of this packing is the amount of global memory available in the GPU; however, STORMM's data structures and algorithms are designed to conserve RAM, whether on the CPU host or the GPU device. Modern desktop workstations and rack servers often have 64 GB or more of main memory, whereas GPUs with more than 24 GB of their own RAM are available (although still expensive as of the time of this publication). The tests that follow will show STORMM running collections of many individual systems, which, combined, total over 2.6×10^6 atoms. For some applications, a GPU with 10 GB of memory can run problems comprising as many as 7.5×10^6 atoms.

A. The `AtomGraph`

STORMM's basic topology is the `AtomGraph`, containing stiffness constants for valence interactions, atom indices enumerating the valence terms, non-bonded attributes of atoms, names,

05 September 2026 02:57:16

© Author(s) 2024

TABLE I. Properties of STORMM coordinate classes.

Class	Contents[a]	Data type	Bytes per atom	Coordinate sets	Topologies
CoordinateFrame	**r**	float64_t	24	One	One
PhaseSpace	**r**, **v**, **f**	float64_t	144[b]	One	One
CoordinateSeries	**r**	float32_t, float64_t, int16_t, int32_t, int64_t	6–24	Many	One
PhaseSpaceSynthesis	**r**, **v**, **f**	Fused int64_t + int32_t	216[b]	Many	Many
Condensate	**r**	float32_t or float64_t	12 or 24	Many	Many

[a]Cartesian data: **r** = particle positions, **v** = velocities, **f** = forces.
[b]The `PhaseSpace` and `PhaseSpaceSynthesis` objects have twice the expected size for their data types due to holding two copies of all coordinates: one for holding the current or reference state, and the other for constructing the forthcoming state during dynamics or energy minimization.

and other descriptors of the system. In its present state, STORMM reads the AMBER[29] topology file format, with various extensions added by the CHARMM[30] community. Built on `Hybrid` objects, the `AtomGraph` is inherently staged in memory accessible to both the CPU and the GPU. For template compatibility, internal arrays storing all floating-point data such as partial charges or dihedral potential amplitudes are copied as `float32_t` and `float64_t` values.

B. Specialized coordinate representations

STORMM has five unique coordinate objects, including an object that only stores Cartesian x, y, and z particle positions with box dimensions (if applicable), a simple representation of particle positions, velocities, and forces that is suitable for managing molecular dynamics of a single system, and an extensible array of coordinates for a series of snapshots from a single system. While all box dimensions are held in `float64_t` (`double` in most C++ programs), the data types of coordinates in each object can vary based on its purpose. Constructors for each coordinate class accept the format for underlying `Hybrid` objects so that a developer may create host-bound or GPU-only coordinate data, rather than the default plan of mirroring the allocations on both resources, to conserve memory in data-intensive applications.

Traffic between all coordinate objects is managed by a versatile API, which includes `coordCopy()` to copy coordinates between any two coordinate objects, conserving as much bitwise detail as possible and altering only memory on the CPU host or GPU device as specified in the call, e.g., coordinates may be taken directly from the GPU-based memory of object A to the CPU host memory of object B, without altering the host memory content of A or the GPU-based content of B. For like coordinate objects each containing multiple structures, the `coordSwap()` function can swap one or more systems from one to the other, to and from memory on either the host or the device, with a single kernel launch to maximize communication bandwidth. Table I lists specifications of the available classes, while Table II indicates their possible applications.

C. Many coordinate systems in one: The `PhaseSpaceSynthesis`

The most complex, memory-intensive, and secure coordinate object is the `PhaseSpaceSynthesis`. Like the primitive `PhaseSpace` object, it stores positions, velocities, and forces, but it concatenates like pieces of the information for each system, e.g., the particle positions along the Cartesian x axis, and stores arrays of limits to track the demarcations between each system. Designed for GPU computing with bitwise reproducibility, the `PhaseSpaceSynthesis` stores all coordinates in the fixed-precision format with combined `int64_t` and `int32_t` data. The length of the fixed precision representation for each coordinate type may be specified with the creation of the object, but any aspect of the phase space and forces for each system may be specified to a precision of up to one part in 2^{72} of the internal units of Angstrom (Å), Å per femtosecond, and kcal/mol-Å. For convenience, the `PhaseSpaceSynthesis` object also stores pointers to the original topologies for each of the systems in its collection. Various constructors are available to create the `PhaseSpaceSynthesis` from an array of existing `PhaseSpace` objects, replication, and other indexing or tiling methods. There is no restriction on disparate system sizes present in any one `PhaseSpaceSynthesis`, but there is a

TABLE II. Uses of STORMM coordinate classes.

CoordinateFrame	High-precision storage of a single snapshot; data analysis; structure manipulation; CPU-intensive calculations
PhaseSpace	High-precision storage of a molecular system; propagation of a single molecular dynamics simulation, likely in non-deterministic fashion; CPU-intensive calculations
CoordinateSeries	Variable-precision storage of a simulation trajectory; construction of multiple initial states; buffered output for simulation frames prior to collective download and disk writeout; data compression
PhaseSpaceSynthesis	High-precision handling of diverse molecular systems; bitwise-reproducible parallel or GPU-enabled calculations
Condensate	Staging frames from a PhaseSpaceSynthesis in floating-point data; snapshot and trajectory analysis; common wrapper for either PhaseSpaceSynthesis or CoordinateSeries objects

05 September 2026 02:57:16

© Author(s) 2024

restriction that either all systems have periodic boundary conditions or no system has any boundary conditions.

D. Many topologies in one: The `AtomGraphSynthesis`

Just as the `PhaseSpaceSynthesis` compiles multiple coordinate systems by concatenating their component arrays, the topology synthesis, `AtomGraphSynthesis`, compiles multiple `AtomGraph` topologies describing different systems into a single object, concatenating arrays such as atomic partial charges, Lennard-Jones tables, and bond term atom indices in the process. In the interest of conserving local cache space, STORMM will make consensus tables for valence terms such as dihedral parameters so that all systems can work from common information that is likely to be relevant even if the molecular systems themselves differ. However, Lennard-Jones pairwise interaction matrices, which would grow as the square of the number of unique atom types, are only used for replicas of the same system: otherwise, the Lennard-Jones pairwise matrices for each unique topology are concatenated into a common array. Tracking systems and interactions in the `AtomGraphSynthesis` is a complicated task, but this is handled by single-threaded C++ code for easy debugging. The C++ code reconfigures atom indices and condenses parameter tables as needed and then constructs work units with compact instruction sets encoded in bit strings so that GPU kernels can perform cycles of work, one of which is detailed in Sec. IV B.

The `AtomGraphSynthesis` emits several abstracts for handling valence interactions, non-bonded interactions, virtual site manipulation, and other calculations depending on the work of a particular kernel.

E. Supplemental restraints: The `RestraintApparatus`

While a complete chemical model can express interactions among particles in the form of the STORMM `AtomGraph` and `AtomGraphSynthesis`, computational experiments often rely on controlled perturbations that can be subtracted from the simulation in post-analysis. STORMM offers the `RestraintApparatus` to encode flat-bottom, harmonic restraints similar to AMBER's nuclear magnetic resonance (NMR) potentials for a chemical system. This is a critical supplement to the basic topology by which developers can create ways for users to "push" or "pull" on selected atoms. Formally, the information in restraints is separate from the topology (the unitary `AtomGraph`), but it can be incorporated into the `AtomGraphSynthesis`.

Due to the difficulty of enumerating restraints over the large numbers of systems that STORMM is designed to handle, users can specify groups of restraints with a single command, e.g., "all heavy atom dihedrals in this atom selection mask" or "distance restraints that penalize intra-molecular hydrogen bonding." Restraints can also be applied to a particular structure, a labeled group of structures, or all systems in the runtime instance.

IV. UNIQUE NUMERICAL IMPLEMENTATIONS

Integers, or the ability to interpret floating point numbers as integers, are fundamental to STORMM's optimizations. Fixed-precision accumulation, critical to reproducible parallel computing, is handled in a unique manner with improvements to speed and memory locality. Section 3 of the supplementary material elaborates on a templated class for emulating functions of a single variable, e.g., the distance between two particles, as a compact tabulated spline. Functions that are very steep at small values of the underlying variable can be reproduced with high fidelity due to the logarithmic indexing technique made possible by interpreting floating point numbers as unsigned integers and taking their high bits. This method was first implemented in the Amber GPU engine,[31] but STORMM introduces more generality and a new way of evaluating the splines, which improves accuracy by twofold to twelvefold. In other contexts, unsigned integers are used as packed strings of bits to encode compressed instructions for individual interactions when computing valence interactions. GPUs have limited cache space and memory bandwidth relative to their arithmetic capability, but STORMM's infrastructure is built around clever data compression methods to recover the balance.

A. Split fixed-precision representations with `atomicAdd()`

One of the most significant numerical innovations in STORMM is the use of multiple accumulators to store and accumulate fixed-precision representations of numbers, in multiple integers. Fixed-precision representations work by multiplying floating point numbers by some power of 2, common choices ranging from 20 to 40, and recasting them as integers to accumulate the result. The rounding technique is a matter of choice, although the goal is often to provide enough information in the fixed-precision number to capture all information in the mantissa of most floating-point contributions. Fixed-precision accumulation can be used to extend the precision of calculations taking place within well-bounded ranges, when catastrophic cancellation is a problem. More often, it is used to achieve bitwise reproducible results in parallel programming, where sequential uncertainty can combine with the non-associativity of floating point addition.

In many situations, it is sufficient to accumulate quantities such as the representation of a force with a fixed-precision scaling of perhaps 2^{24} such that most accumulations will not go outside the range $[-2^{31}, 2^{31})$. However, if an accumulation does stray from this range, even on occasion, the result can be catastrophic: in the case of forces, a large negative force flips to become a large positive force. The solution is to make the fixed-precision representation able to accommodate numbers so large that the simulation would itself be broken if a legitimate value of such magnitude, e.g., force in molecular dynamics, were to appear. The insurance policy can be expensive: if 32 bit integers are sufficient to hold 99.99% of the accumulated forces, then a 64-bit representation, which can hold 100% of the results, while necessary to ensure a stable simulation, is transmitting twice as much information as necessary in all but one case out of 10 000.

A *split* fixed precision representation makes the insurance policy more adaptable: accumulation can go on in the "primary" accumulator, a 32-bit accumulator in the example above, while holding a second 32-bit accumulator in reserve in case of emergencies. Two conditions must be identified: first, when a specific contribution or initialization would overflow the primary accumulator, and, second, when an incremental accumulation out of a series causes the primary accumulator to overflow. In each case, the overflow of the

05 September 2026 02:57:16

© Author(s) 2024

```
// The accumulation for int32_t is 2^31, that for int64_t is 2^63
constant real max_accumulation = pow(2, SizeInBits(integer) - 1);

// The real-valued input can be taken as a constant.  The two
// accumulators will be modified by the function.
atomicSplit(real value, integer primary, integer overflow) {
  integer spillover = value / max_accumulation;

  // Bank rounding is best, but the rounding method is likely
  // to be inconsequential as the floating point number will
  // have been stripped of so many of its high bits that all of
  // its remaining detail fits within the integer format.
  integer ival = RealToInteger(value -
                               (spillover * max_accumulation));

  // Contribute to the primary accumulator.  Collect the return
  // value from the atomic addition.
  integer prim_old = atomicAdd(primary, ival);

  // Test whether the primary accumulator has rolled over.  If
  // so, fold an additional two increments into the contribution
  // for the secondary accumulator (the secondary accumulator
  // deals in units of max_accumulation, half the breadth of the
  // primary accumulator).
  integer prim_old_plus_ival = prim_old + ival;
  if ((prim_old ^ prim_old_plus_ival) < 0 and
      (prim_old ^ ival) >= 0) {
    spillover = spillover + (1 - (2 * (ival < 0))) * 2;
  }
  if (spillover) {
    atomicAdd(overflow, spillover);
  }
}
```

FIG. 2. Pseudo-code for split fixed-precision accumulation. Function names are highlighted in gold, and variable types are written in purple. Comments appear in red. The carat (^) indicates bitwise exclusive OR (XOR).

primary accumulator must be contributed to the secondary accumulator. STORMM implements the scheme illustrated in Fig. 2, combining two 32-bit signed integers to support a 63 bit range for accumulating results from 32-bit floating point math or one 64-bit primary accumulator and a 32-bit overflow accumulator to support a 95 bit range for accumulating results from 64-bit floating point math.

As split accumulation is used throughout the STORMM code base, the performance enhancements it confers will be presented in context. For reference, a handful of benchmarking kernels were created that perform a minimal amount of math to vary `float_32t` counters with a median value of 45.0 ± 17.0, then scale them by different fixed-precision factors, and accumulate them using atomic operations. In these cases, the vast majority of the work lies in the atomic accumulation and, depending on the depth of the fixed precision accumulation, the primary accumulator overflows with different probabilities, triggering the secondary atomic operation. Figure 3 shows the significant advantages that dual `int32_t` accumulators can have over a single `uint64_t`, particularly on commodity cards when the accumulators reside in a `__shared__` memory partition of the chip's local cache.

When the sum rarely spills into the secondary accumulator, the pair of `int32_t` accumulators can work up to twice as fast as a unified 64-bit accumulator depending on the card model. However, for the professional-grade A100 GPUs, the performance of 64-bit accumulation in `__shared__` resources appears to be as fast as solitary 32-bit accumulation, supplanting split accumulation by a small amount. On all cards, the cost of each atomic reaches a plateau as the accumulations approach 2^{31}, the maximum amount that the primary accumulator can accept in a single operation and also the most likely to trigger incremental overflow. The split accumulation carries an advantage over direct 64-bit accumulation in `__global__` memory when the secondary accumulators reside there (atomic operations to global memory addresses take place in the global L2 cache), and even, in the case of the A40 GPU, when both accumulators reside there. Keeping the primary accumulator in `__shared__` while storing the secondary in `__global__` is a way to conserve the most valuable chip cache space while still protecting against catastrophic overflow. Of course, the split accumulators can be used to carry out fixed-precision calculations on cards that do not support 64-bit atomic operations, for which the OpenMM program has adopted a similar method.[32]

While it is safe to add any two split-fixed precision numbers using the method in Fig. 2, it is not safe to multiply a split fixed-precision number's two components by −1 in order to get its negative value. This arises from the asymmetric nature of inte-

05 September 2026 02:57:16

© Author(s) 2024

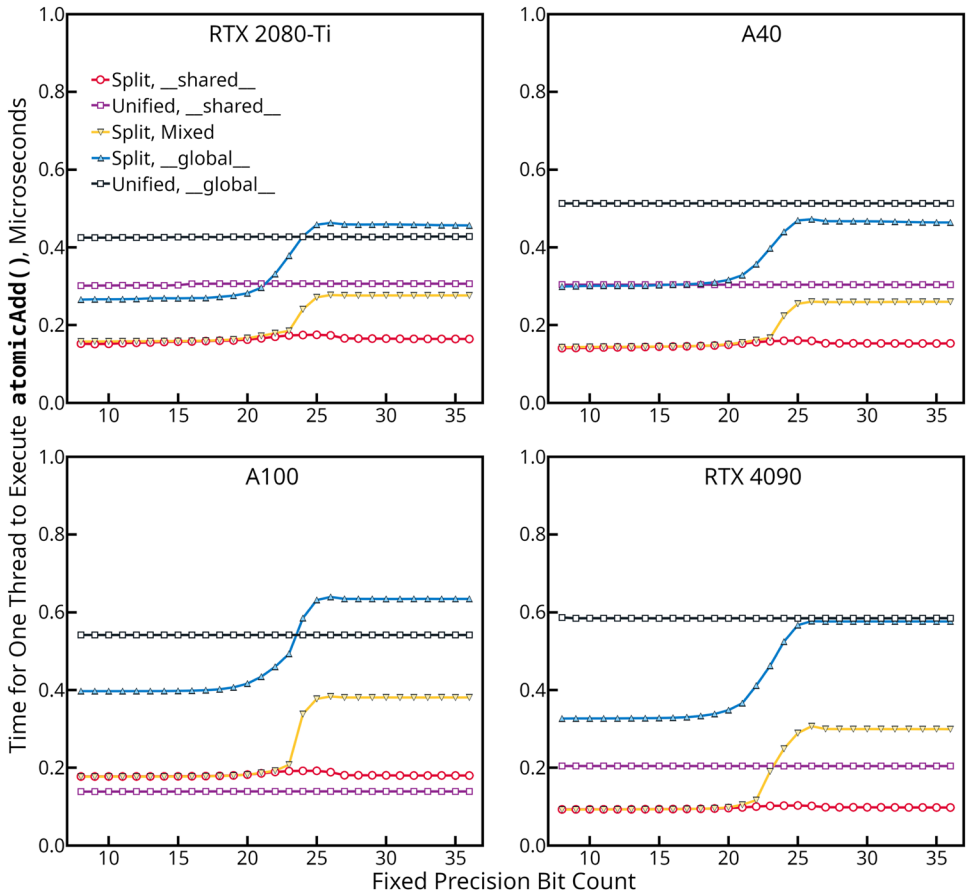


FIG. 3. Timings for coalesced, non-competing atomics. In each operation, a `float32_t` number is scaled by a power of two indicated by the number of fixed precision bits, converted to integer format, and added to accumulators within the local or global cache. The kernel launch grid involved 1024 threads per streaming multiprocessor on each GPU. The overall throughput of any card is a function of the speed with which one thread can execute each atomic and the total number of threads active across all streaming multiprocessors. The inset legend in the upper left panel applies to all panels.

gers represented in two's complement. The largest negative value an integer can hold is one larger than the largest positive value. Therefore, multiplying the largest negative value by −1 overflows the largest positive value by one, causing the number represented in the computer to roll back around to the largest negative value. To a typical CPU or GPU computer, `-INT_MIN = INT_MIN`. This implies that a split fixed-precision number whose primary accumulator holds the largest negative integer must be flipped in sign by taking the negative of its secondary accumulator and adding 2 to that (the result can be submitted to the same accumulation shown in Fig. 2). This oversight was corrected in review in response to a request for investigation into a crash that occurred in one simulation out of 1280 runs, each exceeding 8 μs (see Sec. V C). To prevent developers from repeating the subtle mistake, STORMM includes functions to subtract one split fixed-precision number from another, folding in the safeguard.

Some notable properties of the GPUs presented in Fig. 3 are listed in Table III. While the speed of split atomics to `__shared__` appears to be a function of the cards' clock speeds, the trend with more impact may be the steady decline in the speed of 32-bit as well as 64-bit `atomicAdd()` instructions to `__global__` memory addresses. In a given amount of time, the RTX 4090 can issue more instructions to L2 than an RTX 2080-Ti because the 4090 has almost twice as many streaming multiprocessors, but the instruction bandwidth falls relative to the computing capacity from one generation to the next. The benchmarking program used to collect the data presented in Fig. 3 is distributed with the STORMM code base, as are other small programs used to carry out most of the studies in this paper.

The precision model in STORMM is intended to be very flexible and, with sensible default parameters, in the user's control. Rather than simply applying "single" or "double" precision variants of the program, users will be able to apply a `&precision` namelist to their molecular simulations to toggle between single and double precision in various aspects of the calculations and also control the fixed precision bits used to accumulate forces or represent the phase space.

05 September 2026 02:57:16

© Author(s) 2024

TABLE III. Critical parameters for NVIDIA GPUs used in this study.

Card line	CC[a]	SM[b] count	Clock speed (GHz)[c]	Global memory (GB)	Global memory bandwidth (GB/s)	Total L2 cache (MB)	Total L1 cache per SM[d] (kB)	Power draw (TDP) (W)
RTX 2080-Ti	7.5	68	1.35/1.55	11	616	5.5	96	250
A40	8.6	84	1.31/1.74	48	696	6	128	300
A100	8.0	108	1.07/1.41	80	1935	80	192	300
RTX 4090	8.9	128	2.23/2.52	24	1008	72	128	450

[a]The compute capability of the architecture, as codified by NVIDIA.
[b]Streaming multiprocessors (SMs) contain compute units within the graphics card. Each SM has its own L1 cache resource and arithmetic units.
[c]The cards' base/boost clock speeds are displayed.
[d]The total L1 capacity is taken as the sum of texture and `__shared__` configurable resources.

Continued development in STORMM will keep all options open in kernels that rely on a great deal of communication, with the goal of fulfilling any requested precision model in the most efficient and safe manner possible.

B. Advanced work units for valence interactions

In a CPU-based simulation, particularly running at low parallelism, the valence interactions are a trivial calculation, about 2%–4% of the overall wall time. Here, "valence interactions" denote all bond, bond angle, dihedral, CHARMM Correction Map (CMAP),[24] and attenuated non-bonded interactions (usually between atoms separated by three molecular bonds). The number of interactions scales in proportion to the number of atoms connected by such features. Water, the most common component of many simulations, has no dihedral angles (the most numerous valence interaction and one of the most laborious to compute) and often no flexible bonds. However, each of the terms can involve a large number of memory transactions to obtain coordinates and parameters, a fact that, in naïve implementations, favors CPUs over GPUs. The ease with which CPUs can accomplish valence interactions has led some GPU programs[11] to support computing paradigms whereby all coordinates are transferred to the host RAM for these and other memory-intensive calculations, e.g., constraint algorithms, to proceed. However, even naïve GPU implementations can be competitive given the latency and bandwidth requirements of downloading coordinates, then uploading forces, through the machine's PCIe bus.

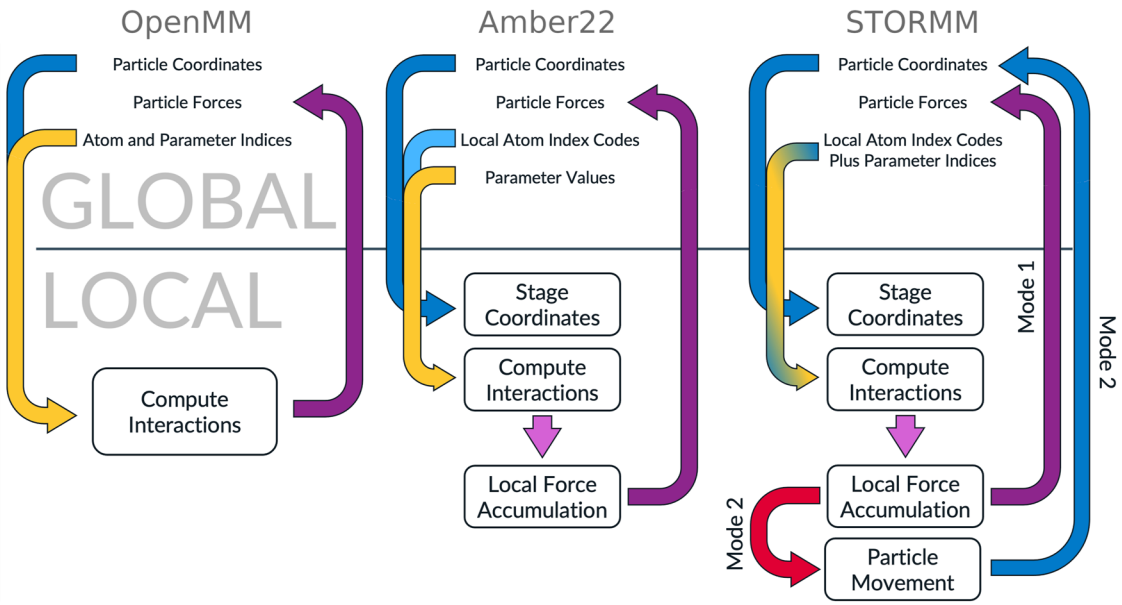


FIG. 4. Workflow for valence calculations in OpenMM, AMBER22, and STORMM. Transfers of coordinate data are indicated by blue arrows, and those of parameter data are indicated by yellow arrows. Purple arrows connote atomic accumulation of forces. Locations of various actions and data arrays are divided between local (on chip) and "global" memory. Staging coordinates and local force accumulation are, broadly speaking, examples of explicit data caching. Implicit caching effects are important to OpenMM with coordinates and to both OpenMM and STORMM with parameter tables, although neither is shown on the chart. STORMM's valence work units can function in either of two modes to serve different molecular mechanics calculations.

05 September 2026 02:57:16

© Author(s) 2024

Optimizations in the AMBER code[31] included a set of work units, created by several thousand lines of C code running on the host after the topology was first loaded, to improve the performance of this aspect of MD simulations. The strategy involved stepping across the lists of valence interactions and adding atoms as needed to a short list (maximum 128 atoms). As more atoms were added to fulfill one interaction, the list was checked for other interactions that might also be fulfilled. As the buffer of atoms filled up, additional interactions seeded new lists. A "work unit" then consisted of a collection of atoms to import into local chip cache on some GPU streaming multiprocessor followed by a series of instructions for the valence interaction type, its parameter indices, and the local indices of cached atoms it applied to (see Fig. 4). Evaluating all the covered interactions would accumulate forces and energies, also in the local cache, for a buffered write back to the `__global__` force tallies at the end of the work unit assignment. The strategy was found to expedite dynamics in AMBER18 and beyond, although the benefits were greatest in large systems and small protein-in-water simulations such as the dihydrofolate reductase benchmark (23 558 atoms) did not gain (or lose) much speed. Even with the caching optimizations, the kernel was found to be memory-bound, though not in as severe a way as the earlier, naïve method in AMBER16 (direct queries of all atoms and parameters for each interaction).

Table IV shows that the AMBER code still spends a considerable amount of time processing valence interactions, and on systems of 100 000 atoms or more, the modern GPUs continue to perform in proportion to the bandwidth of their `__global__` memory buses. Timings were obtained by forcing each code to repeat the

TABLE IV. Timings for AMBER and OpenMM 8.1 running four AMBER benchmark MD systems. Run conditions: 4 fs time step, constant volume, temperature maintained at 300 K by a Langevin thermostat, 9.0 Å cutoff on short-ranged electrostatic and Lennard-Jones interactions, and TIP3P water. PME was applied only to electrostatics, with each program's default behavior determining the grid density. Overall timings for each time step were compiled with a post-release modification to 8.1 (see the supplementary material, where timings in ns/day are available), but the valence calculation timings will be consistent with the 8.1 release version.

System	DHFR[a]	Factor IX[b]	Cellulose[c]	STMV[d]
		RTX 2080 Ti		
AMBER μs/step	414	1148	5285	15 319
OpenMM μs/step	296	1104	6380	19 893
AMBER μs/V.C.[e]	42 (10.2%)	46 (4.0%)	400 (7.6%)	824 (5.4%)
OpenMM μs/V.C.	12 (4.2%)	28 (2.0%)	309 (4.9%)	704 (3.3%)
STORMM μs/V.C.	22	38	213	433
		A40		
AMBER μs/step	388	940	4138	11 711
OpenMM μs/step	238	765	5326	15 947
AMBER μs/V.C.	46 (11.8%)	47 (5.0%)	325 (7.8%)	652 (5.6%)
OpenMM μs/V.C.	11 (4.5%)	25 (2.2%)	238 (4.5%)	573 (3.2%)
STORMM μs/V.C.	14	24	134	301
		A100		
AMBER μs/step	312	690	2372	6669
OpenMM μs/step	256	697	2689	8195
AMBER μs/V.C.	15 (4.7%)	16 (2.3%)	62 (2.6%)	109 (1.6%)
OpenMM μs/V.C.	12 (4.5%)	23 (2.1%)	176 (6.6%)	342 (3.2%)
STORMM μs/V.C.	13	25	57	148
		RTX 4090		
AMBER μs/step	250	322	1863	5000
OpenMM μs/step	162	419	1737	5270
AMBER μs/V.C.	43 (17.4%)	28 (8.6%)	158 (8.5%)	302 (6.0%)
OpenMM μs/V.C.	9 (5.5%)	13 (2.0%)	160 (9.3%)	317 (4.9%)
STORMM μs/V.C.	9	18	58	125

[a]Dihydrofolate reductase, 23 558 atoms (including 21 069 in water).
[b]Factor IX, 90 906 atoms (including 85 074 in water).
[c]Cellulose fibers (in water), 408 609 atoms (including 317 565 in water).
[d]Satellite tobacco mosaic virus, 1 067 095 atoms (including 900 159 in water).
[e]Each code's cost of valence interaction computation is likewise given in microseconds. The percentage of time spent in this section of the code is given in parentheses for complete MD packages.

05 September 2026 02:57:16

© Author(s) 2024

valence calculation many times, inserting the extra work after forces had been applied to move atoms and just before the buffers were cleared to ensure that the dynamics remained stable. The cost of the particular kernel was isolated from the differences in wall clock time needed for the original and modified codes to run a given number of steps, a technique that will be used again in Sec. IV C.

STORMM uses an approach inspired by the AMBER design, with critical refinements. The work units are laid out based on graphs of contiguous atoms, not passes over specific valence terms. Figure 4 compares the approaches. In AMBER, each work unit is expected to contribute its accumulated forces back to `__global__` tallies, but in STORMM, they may either follow a similar track or become the capstone of the force calculation, reading the results of prior non-bonded computations and then updating the positions of the atoms they have already cached. Constraints, virtual site placement, and even the thermostat may thus fuse with the valence interaction kernel, saving launch latency and eliminating the need for particles to make multiple round trips between the card's RAM and the streaming multiprocessors. In order to cover all valence interactions, there is overlap between work units: each work unit comes with a mask of atoms that it is responsible for moving. In the most complex cases, such as virtual sites with frame atoms subject to their own constraints, each work unit understands an order of operations and comprises all atoms whose positions and total force contributions must be known in order to complete its position update assignments. Some valence interactions are computed by more than one work unit in STORMM, in the interest of having the results for force transmission from virtual particles and the final coordinate update. If required, energies for such redundant calculations are likewise tallied only by one work unit responsible for doing so, according to the plan laid out on the CPU after loading the topology.

As is evident in Table IV, the STORMM valence work units are superior to the AMBER implementation, as is OpenMM's implementation except when AMBER can work with a strong memory bus. AMBER's implementation lessened but did not eliminate the memory bottleneck associated with valence evaluations. While it caches atoms and performs local accumulations of their forces, the process of computing each dihedral term still involves reading a 64-bit instruction containing the atom indices followed by copies of the force constants (amplitude, periodicity, and phase angle). The second-most numerous type of valence interactions, attenuated "1:4" non-bonded interactions, likewise involve import of new values for each pair. In contrast, STORMM's 64-bit instructions (the dihedral is illustrated as an example in Fig. 5) compress the data much further and use additional indices into parameter tables that the kernels take care to conserve at various levels of cache. STORMM exploits the strong association between dihedral terms and 1:4 attenuated non-bonded interactions to reduce the number of instructions and `atomicAdd()` operations they entail. Furthermore, STORMM exploits the fact that many dihedral terms apply to the same quartet of atoms, differing only in amplitude, periodicity, and perhaps the phase angle. In such cases, or to accommodate bespoke force fields with more than 65535 unique dihedral parameters, STORMM's dihedral instructions can refer to a secondary 32-bit instruction encoding another cosine series term. With 8 bytes read from `__global__` (and not cached, as it is used once per time step), STORMM obtains information that may take 48 bytes to convey in the AMBER work units, or 32 bytes in OpenMM's scheme. In its extended 12 byte dihedral instructions, STORMM transmits information that would take 76 bytes in AMBER and 52 bytes in OpenMM.

OpenMM compiles a unique kernel for evaluating bonded interactions in the simulation at hand, although only the overall

05 September 2026 02:57:16

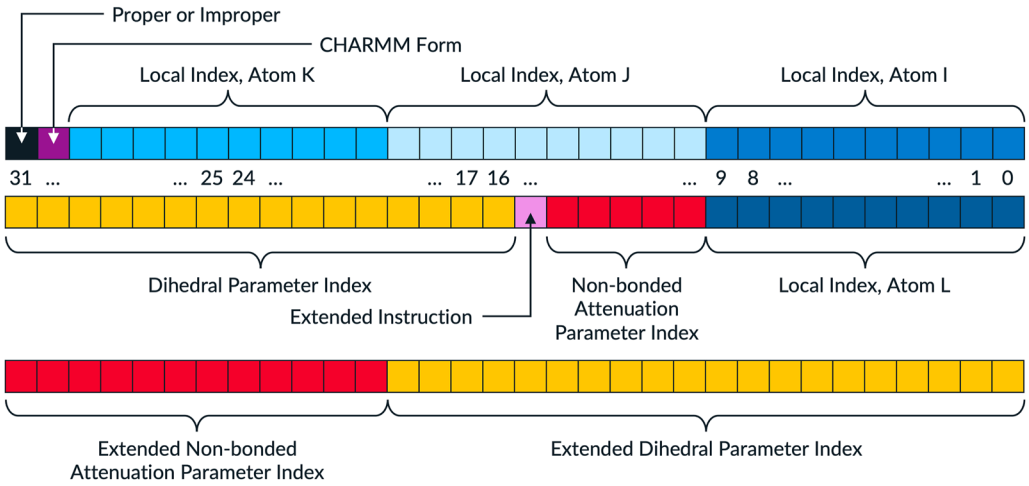


FIG. 5. Map of the bitmask in a STORMM dihedral instruction. The basic dihedral instruction consists of 64 bits, including 40 for atom indices (I, J, K, L) in the local cache, 16 for the term's parameter index (a code of `0xffff` in these bits indicates zero amplitude), and 5 for the I:L non-bonded attenuation strength (a code of `0x0` in these bits indicates no non-bonded interaction). Other bits are used to differentiate proper and improper dihedrals (STORMM logs these energy components separately), and whether an additional cosine term should be applied to the same four atoms (dihedrals with three or more terms in the cosine series would entail new instructions targeting the same atoms). The extended instruction accommodates up to a million unique dihedral parameters and 4096 unique non-bonded attenuations.

© Author(s) 2024

numbers of each term are hard-wired. (The benefit is that only those terms present in the actual model are then compiled, minimizing register pressure by omitting terms with complex expressions when they are not used.) This is by far the simplest approach, as shown in Fig. 4. Atom indices and term parameters are still imported from data arrays in `__global__`, making it much like AMBER's original GPU implementation, although it lessens memory traffic by referring to parameter sets by their unique indices and leaning on global caching effects for atomic coordinates. It may be impossible to exceed the speed of this approach for very small systems with any model based on work units, which require various synchronizations across the thread block in order to have local force accumulation, even if the overall throughput benefits at higher atom counts. AMBER's work unit scheme relies on a feed of raw parameters for each term, e.g., two `float64_t` for a bond's equilibrium and stiffness constants, rather than exploiting indices and parameter tables. This was one of the major changes in STORMM that compressed the data stream. In that sense, OpenMM's implementation has features of both AMBER and STORMM, and the timings reflect it.

While all codes do the bulk of their work in `float32_t` arithmetic and accumulate in some sort of fixed-point scheme, the precision models differ. This carries consequences for the required memory bandwidth or floating-point arithmetic throughput and the overall speed of the kernel. Table V compares the methods in each code. Both the AMBER and OpenMM codes are able to produce microcanonical ("NVE") dynamics with very low energy drift over long simulations, but for its combination of precise displacements, high-precision accumulation throughout, and safeguards against instabilities in `acosf()` for dihedral angles near 0 or π, STORMM is likely to be the most stable of the three codes in this respect.

The advance in overall speed across a variety of systems is a combination of bandwidth optimization as well as flexible sizing in the work units themselves. While all of STORMM's valence work units must have one consistent maximum size for a given kernel launch, the size is set based on the workload and can vary between 68 and 544 atoms. Likewise, the thread blocks that process each work unit can vary in size between 64 and 512 threads. STORMM will query the GPU and decide how best to cut the problem into pieces that will keep as much of the card occupied as possible. The benefits also extend to STORMM's ability to run multiple systems in a single runtime instance. By design, each valence work unit tracks only one set of energy terms and takes a single offset for atom indexing into the topology and coordinate syntheses. Therefore, each work unit applies to at most one system, even if there remains space for many more atoms, but in the case of tens of thousands of small molecule problems, the smallest work unit division can still achieve good thread utilization. When multiple work units combine to evaluate a single problem, the overlap between them is modest, and with very large problems, the largest work units can have overlaps as low as 1%–2% of their overall content (data not shown).

Despite the results in Table IV, STORMM is working against a major disadvantage: its valence work units cover all atoms in each simulation, even rigid water molecules, which have no meaningful bond or bond angle terms and are, therefore, skipped by AMBER's work units and OpenMM's bonded interaction kernel. STORMM

05 September 2026 02:57:16

TABLE V. Features of the valence interaction computation in three code bases. Each code includes a run mode focused on `float32_t` "single" precision. These features apply to that mode; all code bases also include a "double" precision mode, which raises accuracy and cost of most facets of the calculation.

Code base	AMBER	OpenMM	STORMM
Displacements[a]	float64_t	float32_t	int64_t, 28–40 bits[b]
Computation, bonds, and angles[c]	float64_t	float32_t	float32_t
Computation, other[d]		float32_t	
Accumulation, bonds, and angles[e]	40 bits, __global__	32 bits, __global__	22–40 bits,[b] __shared__
Accumulation, other[f]	20 bits, __shared__	32 bits, __global__	22–40 bits, __shared__
Guard for acosf[g]	No	Yes	Yes

[a]The precision in which the displacement between particles is computed. Once the small displacement is computed from two (possibly much larger) numbers in each precision model, all codes convert the result to `float32_t` or `float64_t`, as appropriate to the specific interaction.
[b]STORMM allows the user to select the fixed-precision model for coordinates and force accumulation, within what are deemed to be safe limits.
[c]Floating-point arithmetic used in computations of bond or bond angle terms.
[d]Floating-point arithmetic used in computations of dihedral, CMAP, attenuated non-bonded interactions, and various restraint forces.
[e]The number of bits in the number's fraction is given, followed by the location of the accumulator. Following its local force sums, STORMM is running in a mode whereby the valence interaction contributions are added back to `__global__` accumulators stored in `int64_t`.
[f]The method of accumulating forces emerging from all other interactions.
[g]The arccosine function becomes unstable in `float32_t` computation for arguments leading to angles near 0 or π. The remedy in OpenMM and STORMM is to detect such situations and evaluate (or approximate) the angle with arcsine.

© Author(s) 2024

is anticipating that these atoms will be subject to constraints if the molecules themselves are not flexible, or that the atoms may need to be moved (both processes fall within the purview of the kernels that evaluate valence interactions). To evaluate each code in strict terms of valence interaction throughput, a basic unit of di-, tri-, and tetra-peptides, 2000 atoms in all, was assembled and compacted into a cubic box about 27 Å on a side at 1 atm pressure. This protein-like fluid was tiled to varying degrees to create systems with up to 500 000 atoms bearing the sort of valence interactions found in typical biomolecules. No atom movement or constraint applications were requested in the STORMM kernel's calculations, to be consistent with the established codes' kernels: STORMM was set to operate in "mode 1" illustrated in Fig. 4. The same experiment behind Table IV was repeated for the range of systems to time each implementation. The results are presented in Fig. 6: when measured as a function of the total number of protein atoms, STORMM shows the strongest performance overall, rivaled only by AMBER's on the A100. While OpenMM's wall time rebounds on some lower-bandwidth cards as the system size gets very large, given the logarithmic scale of the plots it is unlikely that AMBER's implementation will ever surpass it on such hardware (the largest of the tiled systems contains three times more "protein" atoms than the mammoth STMV simulation in Table IV). STORMM's approach also comes close to equalizing the older graphics-oriented GPUs and the A100, despite the disparity in memory bandwidth, and can muster the tremendous compute power of the RTX 4090 to produce even

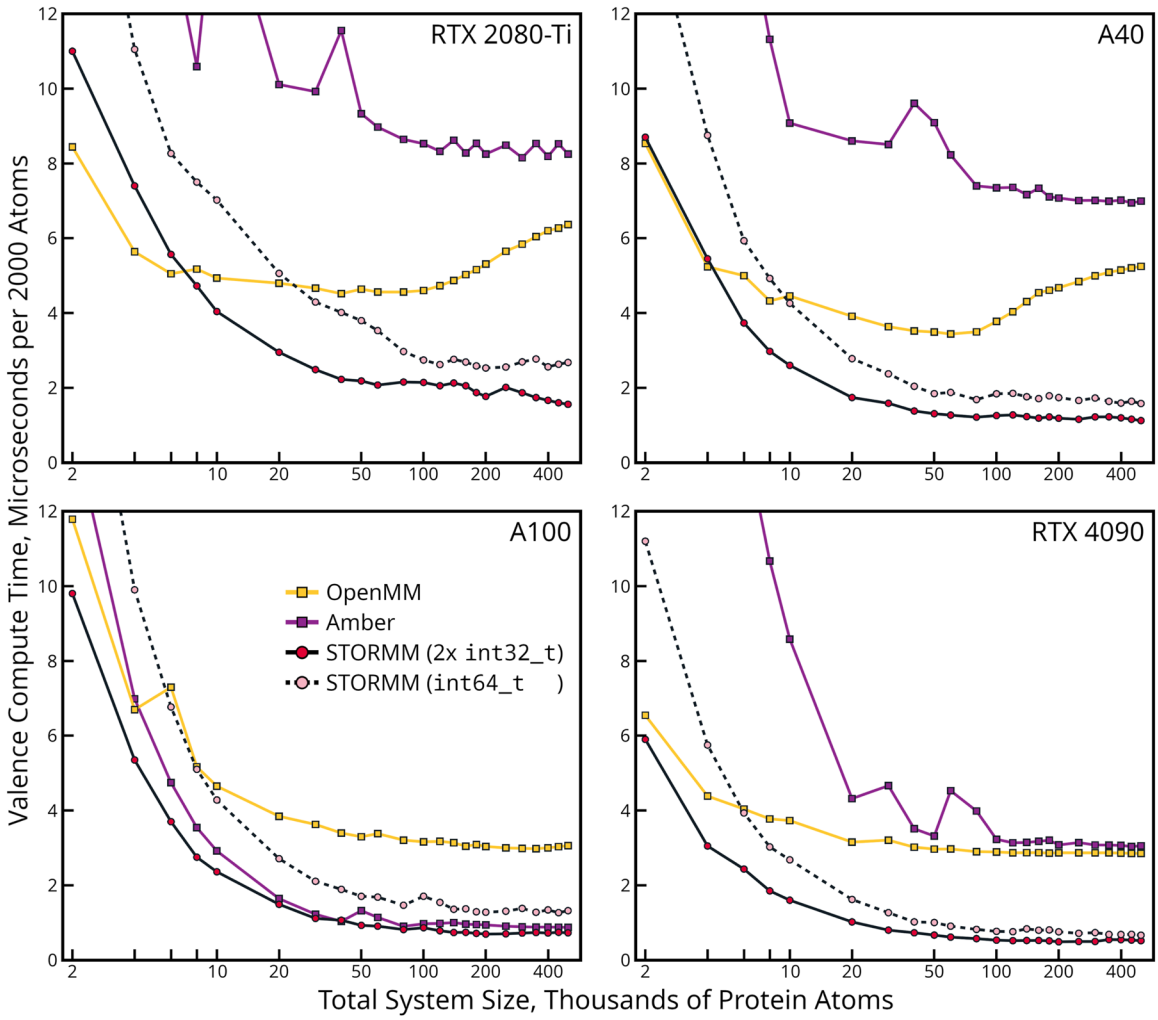


FIG. 6. Comparison of three implementations for valence interactions as a function of total atom count. A molecular system consisting of 2000 atoms in various peptides was tiled as indicated on the *x*-axis to create specific numbers of "protein" atoms bearing valence interactions typical of biomolecular simulations. Each code's valence interaction computation time is plotted per tile of the basic unit to show its scaling with the overall system size. The inset legend in the lower left panel applies to all panels. These unphysical systems stand in contrast to the solvated test cases in Table IV, to create systems where all atoms bear valence interactions. *y*-axis works in time per 2000 atoms as this is about the smallest meaningful increment for measuring the cards' performance: the minimum kernel execution time in any of the codes is around 6–10 μs, and any of the codes would process 1000 atoms in the same time it takes to process 2000.

05 September 2026 02:57:16

© Author(s) 2024

higher throughput (the time to compute valence interactions in a large system is ~0.5 μs per 2000 atoms on an RTX 4090, vs ~0.7 μs on an A100, and ~1.2 μs on an A40). The split fixed-precision accumulation detailed in Sec. IV A shows consistent benefits, even on the A100 (on this card, the split accumulation was not as fast as the unified `int64_t` approach in the benchmark presented in Fig. 3, perhaps because in the prior test no threads were competing for the same memory).

While it is clear that STORMM's implementation depends less on memory bandwidth, the result for A100 in Fig. 6 calls into question other "macroscopic" features of each approach. AMBER is launching three 128-threaded blocks per streaming multiprocessor (SM), a total of 384 threads. Up to Volta, it was found that AMBER's memory-bound kernel did not improve in performance when launching more threads, although the late-generation A100 may provide yet higher returns if it launched 512 or 768 threads per SM. In contrast, STORMM benefits from launching 1024 threads per SM, the maximum possible number given the 2^{16} available registers, yet it is barely faster than AMBER if memory bandwidth is plentiful. The simplest explanation is that STORMM is doing more with more. STORMM's check for numerical stability in `acosf()` and processing of redundant terms for the sake of having complete forces on particular atoms no doubt take a toll. More important is that while AMBER is accumulating the bulk of its calculations in `int32_t` with coarse 20-bit fixed precision, using "fire-and-forget" atomics with no requirement to check the return value, STORMM is using somewhat higher precision and checking for catastrophic overflow with each atomic addition. As will be seen at the end of Sec. IV C and the associated Fig. 9, waiting on the return value of `atomicAdd()` is costly, albeit necessary to provide freedom in the precision model. The fact that split accumulation makes such a dramatic improvement over direct accumulation in `int64_t` (comparable to the effect in Fig. 3) shows how much of the process lies in keeping the tallies.

Valence work should not be a major effort in biomolecular simulations, and the problem is compacted to an appropriate degree in most OpenMM simulations. AMBER's work units, while a step in the right direction, remain dogged by excessive `__global__` traffic due to the layout of some basic parameter arrays. In applications such as membrane simulations, as much as 25% of the total atoms may be found in lipid or protein molecules. It pays to optimize this facet of the calculation lest the code become reliant on the strength of the card's memory bus. In STORMM, a special emphasis has been placed on this design to scale small molecule calculations such as those discussed in Sec. V A, where the valence interactions can account for more than half the GPU wall time. The versatility of the result will benefit much larger condensed-phase simulations as well.

C. Particle–mesh interactions in periodic simulations

The complete periodic simulation capability in STORMM is unfinished, but a strong functionality for one of the most memory-intensive operations is prepared and, like the valence interactions, showcases a powerful strategy in having the CPU examine the workload at hand in order to subdivide it into balanced work units that can achieve high thread utilization on the GPU. The splitting between particle–particle and particle–mesh interactions is described in more detail in Sec. 3 of the supplementary material, in the presentation of a tabulated spline that expedites some of the calculations. The particle–mesh interactions in a molecular simulation follow the same steps as particle simulations in other branches of computational physics:[33]

- Interpolate particle density from the particles' actual positions to regular points on a three-dimensional mesh.
- Perform a "forward" Fast Fourier Transform (FFT) to render the particle density in frequency space.
- Carry out another forward transform of the influence function of one mesh point on all others. This step can be skipped if the influence function has an analytic form in frequency space, as it does in the case of PME.
- In frequency space, multiply all elements of the particle density with the mesh influence potential.
- Complete the convolution with a reverse FFT of the above product, obtaining the mesh potential based on the density at every point projecting its influence onto all others.
- Map the mesh-based potential back to the actual particles based on the interpolation used in the first step. Derivatives of the interpolant indicate the forces on each particle along the mesh axes, which can be transformed into Cartesian space using chain rules.

Of these steps, the FFT is oft reviled as the most difficult to scale. In principle, this is where the global communication occurs, but the FFT accomplishes a tremendous amount of work at an astonishing economy in arithmetic. The influence of one point on the lattice can be extended to all others by computations involving all points directly to the left or right, then all points directly forward or behind, and last all points directly above or below: all-to-all interactions processed between every pair of N^3 points for communicating just $3N$ pieces of information (twice). While it is desirable to minimize global communication requirements, the only way in which this can happen is to reduce the density of the mesh on which global interactions occur, and many alternatives to PME[34–36] still perform FFTs on a mesh spanning the entire periodic simulation cell. On GPUs, the greatest obstacle to reducing the global mesh density in canonical PME is the communication required to map particle density to the mesh. Each particle transmits M^3 density contributions for an interpolation order of M (the reverse process, force retrieval, in principle involves the same communication, but it consists of reads rather than atomic writes). PME and other approaches use stencils based on B-splines to project the particle density. Details may be found in the literature.[25]

As we show in Fig. 7 (compare with Fig. 4 of the supplementary material), this mapping cost can be greater than the combined cost of the forward and reverse FFTs. In that figure, a kinase simulation of 52 889 atoms (orthorhombic unit cell dimensions 85.2 Å by 90.6 Å by 81.9 Å) is tiled (taking the most compact arrangement with the given number of replicas) to explore the effect of system size on particle–mesh mapping in the GPU-enabled MD codes of AMBER22 and OpenMM 8.1. In each case, the particle–mesh mapping cost was measured by nesting a loop inside the main dynamics loop to reset the density mesh accumulators to zero, launch the accumulation kernel, then re-arrange, and (if needed) convert the mesh to `float32_t` suitable for FFTs. Repeating this loop ten times and then comparing the total run time of the adjusted code with that of the original code over 5000–100 000 time steps yielded an esti-

05 September 2026 02:57:16

© Author(s) 2024

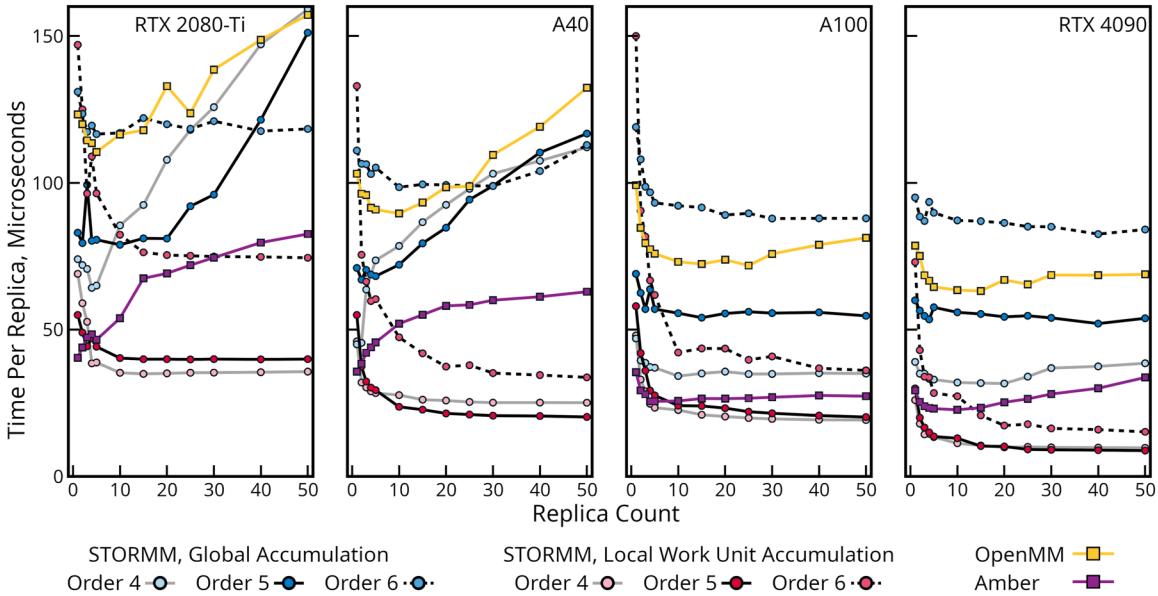


FIG. 7. Time needed to map charge density to PME meshes. Two STORMM kernels are compared (one working in L2 to perform "global accumulation," the other working out of a `__shared__` partition of "local" L1, based on CPU-designed work units) as well as the equivalent code in AMBER and OpenMM 8.1. STORMM kernels can work in a range of interpolation orders, as shown in the legend, and in this test are performing fixed-precision accumulation with 28 bits after the point. AMBER performs fixed-precision accumulation on a similar scale but is restricted to fourth order interpolation. OpenMM runs fifth order interpolation and, for this test, is configured in non-deterministic mode, accumulating in `float32_t` to approximate a similar bandwidth requirement to STORMM and AMBER. The system is a 52 889 atom kinase in water. Mesh discretizations in STORMM were set such that voxels of the simulation close to 4.5 Å in width were filled with 125 points in fourth order interpolation, 64 points in fifth order interpolation, or 27 points in sixth order interpolation, which approximate the mesh densities of equivalent calculations in AMBER and OpenMM. STORMM treats each replica of the system as a separate problem, while AMBER and OpenMM are tasked to simulate a tiled unit cell containing the stated number of copies of the system. GPU properties are listed in Table III.

05 September 2026 02:57:16

mate of the average particle–mesh mapping cost. The OpenMM code also includes a step to sort all particles based on their mesh alignments, which is reused in the force interpolation at the end of the particle–mesh workflow. This sorting cost was measured in the same manner as before, and half was assigned to OpenMM's overall mapping procedure. These timings can be compared to STORMM's two options for mapping particle density to the PME mesh. Both are based on a neighbor list that locates particles to a spatial decomposition grid subdivided by half the particle–particle cutoff distance. Another unique aspect of STORMM is that the PME mesh is aligned to the neighbor list decomposition grid: an integer number of mesh spacings (e.g., 4 or 5) must span each neighbor list decomposition cell in all directions. The first option, "global accumulation," loops over all atoms issuing `atomicAdd()` instructions to the global (L2) cache. In the second (and more advanced) option, a kernel loops over "local work units" spanning the spatial decomposition cells and works within `__shared__` memory resources, a partition of the local (L1) cache on each GPU multi-processor. As with the valence interaction work units, the particle–mesh mapping instructions are drawn by C++ routines to optimize cache utilization and load balance. The "local work unit" kernel has nearly the same breadth of applicability and choice of interpolation order as the "global accumulation" kernel, unless an extreme PME mesh discretization like 8 or 10 per neighbor list cell is chosen.

Robust benchmarking that covers the range of relevant use cases and scenarios is hard. One important thing to note in Fig. 7 is that, while the established MD codes are running calculations on a tiled system for the purposes of comparison, in this test, STORMM's kernels are required to treat each replica of the system as distinct, as would be the case in a replica exchange or weighted ensemble simulation (in Fig. 6, all codes were operating on a single unit cell). The traditional codes are, therefore, able to see all atoms and the PME meshes as contiguous in memory, whereas STORMM is respecting barriers between systems and any undersized work units or disjoint memory spaces this may entail. A precise comparison is also elusive due to the high-level configuration of each engine. In the case of AMBER's code, which runs the charge density mapping in the same stream as the rest of the force calculation when performing standard MD, the comparison with our test program driving the STORMM kernels is strong. However, OpenMM defaults to running its memory-intensive density mapping and FFT work in a separate stream from the compute-intensive particle–particle non-bonded work. Toggling this feature showed that the complete MD cycle benefits by as much as 15%–20% on cards with plentiful L2, but on all cards, the effect tends to diminish with system size and with limited L2 it can even become a detriment (data not shown). In any case, OpenMM's streaming applies to the entire PME workflow, from mapping to force interpolation, whereas the present study seeks to

© Author(s) 2024

examine the efficiency of the mapping in the limit of many atoms. The true cost of the density mapping in OpenMM might not be as high as indicated by Fig. 7, but the OpenMM tests were repeated with and without a separate PME stream, making little change in the mapping times or trends (data not shown), and experience from programming the STORMM kernels as well as AMBER's approach also suggests that the benefit of interleaving this particular part of PME would be minor.

Despite a minor handicap, STORMM's mapping kernels perform very well. The "global accumulation" kernel's timings include the required initialization and accumulator conversion steps as performed in AMBER and OpenMM. It is intermediate in speed between the two established codes although its throughput deteriorates the most rapidly as a function of the total number of atoms in play. On the A100, most approaches show a degree of improvement in their throughput as the number of atoms increases—larger workloads are easier to balance among the tens of thousands of threads that each card supports. However, for the RTX 2080-Ti and A40, all L2-based approaches show a degree of regression, e.g., they run ten replicas of the kinase system more than twice as fast as they run 20. This reflects a critical hardware limitation on some cards: the size of the L2 cache. The RTX 2080-Ti and A40 have L2 cache sizes of 5.5 and 6 MB, respectively (see Table III). The typical density in a protein-in-water molecular simulation is about 1 particle per 9.5 cubic Å; for fourth order interpolation, this is a density of about one particle per ten mesh points; for fifth order interpolation, this is a density of about one particle per five mesh points; and for sixth order interpolation, this is a density of about one particle per three mesh points. With each accumulator being 4 bytes, the single replica of the kinase then has a mesh size of about 0.8–2.4 MB. Adding a second replica may approach the limits of the cache; third and fourth replicas can cause it to start thrashing as threads working on different regions of the problem issue instructions to disparate memory addresses. The A100 (80 GB PCIe), a late-generation model with an expansive 80 MB L2 cache, can handle tens of replicas of the system with even the densest meshes before L2 is filled, and even then the massive resource will have a low rate of overturn: the A100's powerful memory bus (which has 170% more bandwidth than the A40) can keep up. It is clear from Fig. 7 that the design of the algorithm, in particular the pattern in which different particles are mapped, matters. While slowest overall, OpenMM's approach, with its careful pre-sorting of particles to optimize the way it cascades across the mesh, shows less performance degradation than either of the other kernels working out of L2 on the A40 or RTX 2080-Ti. The fastest of these L2-based kernels, in AMBER, uses some aggressive rearrangement of the accumulators to optimize coherence of all threads' write instructions. Whereas STORMM's kernel is writing to M^2 cache lines per atom (and perhaps twice that many, if the mapping wraps across the periodic boundary), AMBER's kernel is mapping to as few as M and at most $4M$. There are also redundant calculations of the same B-spline coefficients in the STORMM kernel, assigning M threads to each atom rather than just 1, whereas the AMBER kernel assigns multiple threads to each atom but pools the computations of the B-spline coefficients. However, the optimizations in the AMBER kernel impose limitations of their own, and STORMM's "global accumulation" kernel is meant to serve edge cases that users might request.

The pervasive degradation of performance with higher atom counts, a sort of digital Tragedy of the Commons where each thread going about its work starves the others of a critical resource, challenges the premise of gaining efficiency by stacking related simulations together. The same problem is likely seen in Fig. 6, as

05 September 2026 02:57:16

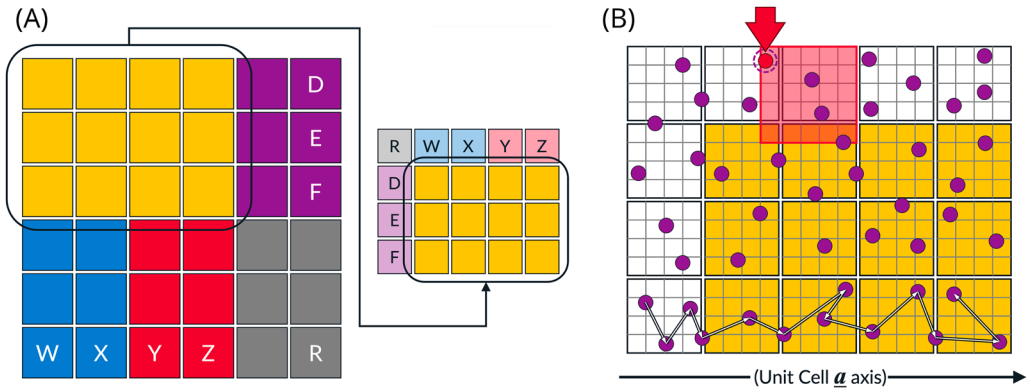


FIG. 8. Two-dimensional illustration of work unit design for particle density mapping in periodic simulations. In panel (a), a periodic unit cell is subdivided into colored work units. The PME mesh regions assigned to each work unit are non-overlapping, but particles in a "halo" region around each PME mesh subdivision must be imported by any given work unit. Wrapping in the periodic boundary conditions is encoded into work units as needed. The three-dimensional implementation in STORMM includes variable heights for each work unit. During setup, work units are arranged in decreasing order of size so that the shortest work units can back-fill the GPU thread blocks and improve load balancing. Panel (b) details an individual work unit, composed of neighbor list partitions (black squares), which align the PME mesh (gray lines). Particles in neighbor list partitions are found in contiguous arrays stepping along the unit cell's *a* axis (the Cartesian *x* axis in an orthorhombic unit cell), but within any given partition, the atoms are expected to be randomized. The work units are designed such that GPU thread groups step along atoms as illustrated by the series of white arrows. The *B*-spline stencil due to one selected particle is marked in red: for an interpolation order of M, a work unit must import atoms in a halo region extending $M - 1$ PME mesh points from its assigned section of the PME mesh [colored gold in panel (b)]. STORMM will detect the card in use and tailor work units to utilize the available L1 cache space. STORMM seeks an *a* axis length of the PME mesh assignment that is not a multiple of 8, 16, or 32, to help distribute work across `__shared__` memory banks.

© Author(s) 2024

OpenMM's L2-focused atom staging and force accumulation begins to show regression on the same cards. To overcome this, STORMM's "local work unit" kernel illustrated in Fig. 8 again pulls the problem closer to the chip and provides the best performance in nearly all cases, except perhaps the smallest number of atoms and the lowest interpolation order, where AMBER is faster on older hardware. Even on the A100, the most favorable circumstances for the traditional L2-focused methods, STORMM quickly overtakes the performance of the AMBER engine running fourth order interpolation with its own fifth order interpolation. On other cards, it is even be able to perform sixth order interpolation in less wall time than AMBER.

New technology can help to a degree. The RTX 4090 has ample cache resources, and L2-based methods indeed show less regression even in very large systems, but none of the L2-based methods run much faster on the RTX 4090 than they did on the A100, or when kept within the L2 capacity of the A40. The L2 instruction bottleneck evident in Fig. 3 may be bearing down. While OpenMM appears to show a modest improvement on this newest card, a closer inspection suggests that most of the difference is in the sorting procedure, which takes 30–50 μs per replica on A40, 30–60 μs per replica on A100, and only 14–35 μs per replica on RTX 4090. In contrast, STORMM's "local work unit" implementation delivers more than double the density mapping rate on RTX 4090 as that seen on A40 or A100. The extreme throughput of this unique implementation, which was tuned on the Ampere cards prior to any tests on RTX 4090, suggests a dividend in "future-proofing."

The "local work unit" kernel's first strength is that it needs no initialization or conversion kernels to support it. The accumulators are initialized in `__shared__` memory and converted to the `float32_t` result, which is then written to `__global__` memory in its complete state. Separate initialization and conversion operations, which can take about 10 μs per replica of the kinase system, account for at least 20% of the run time of STORMM's "global accumulation" implementation. More significant is that the "local work unit" kernel issues `atomicAdd()` instructions to `__shared__` memory, which as shown in Fig. 3 can be much faster than L2. Even with a spacious L2, the next fastest kernel in AMBER only gains about 25%–35% throughput in very large systems, and with limited cache, its throughput can degrade by more than half. In bypassing L2, STORMM's "local work unit" kernel scales with the compute capacity of each card and can more than triple in throughput as more atoms are added.

While the L2 sensitivity of the particle mapping can be mitigated or avoided depending on the algorithm design, the L2 sensitivity of the next phase of the particle–mesh workflow, the FFT, is strong and unavoidable. Details are provided in Sec. 4 of the supplementary material. There will always be memory-bound stages of the particle–mesh calculation, but the innovations in STORMM rephrase the problem as finding the balance of mapping and FFT work rather than limiting mesh-based calculations altogether. By raising the interpolation order, the mesh density can be reduced[37] while conserving the overall accuracy of the calculation. A good rule of thumb is that, to obtain forces of sufficient quality for a stable simulation, PME with fourth order interpolation requires the mesh discretization to be at most 2/3 the σ width of the Gaussian used in the splitting function [the Gaussian width is $1/(2\alpha)$ in terms of the "Ewald coefficient" in Eqs. (1) and (2) found in Sec. 3 of the supplementary material], PME with fifth order interpolation requires that the mesh discretization be at most 4/5 the σ width of the splitting function, and PME with sixth order interpolation can use mesh discretization as wide as the σ width itself. For a direct sum tolerance of 1.0×10^{-5}, a Gaussian σ of 1.5 Å is obtained with a cutof f of about 8.346 Å. Longer cutoffs at the same direct sum tolerance will widen the σ and shorter cutoffs will shrink it in proportion. The default values of the AMBER simulation engines are built around this trade-off: a default particle–particle interaction cutoff of 8 Å with code that tries to get the PME mesh discretization as close to 1 Å as possible, often landing at discretizations of 0.9–0.95 Å. It is an even trade between the interpolation orders: twice the mapping work for about half the overall FFT work, or just over three times the mapping work for a third of the FFT work. A similar analysis applies to the particle–particle cutoff: committing to three times the particle–particle work with a 50% larger cutoff would be another way to reduce the FFT work by a factor of 3 but without changing the amount of density mapping work. In the early days of GPU computing, it was thought that this route might be taken. The fact that the old CPU parameters remain the typical (and fastest) choices for GPU molecular simulations is an indication that not enough has been done to adapt to the communication and memory characteristics of GPUs.

One possible complicating factor is that, with lower mesh densities and each particle issuing more density contributions, `atomicAdd()` instructions will collide on the same memory addresses in much higher rates. Global communication in the FFT is reduced at the price of intense local traffic in the mapping. However, Fig. 7 suggests that, if anything, more intense calculations on a single atom are beneficial, at least up to sixth order interpolation with appropriate mesh densities. In the limit of large atom counts, fifth order interpolation by the "local work unit" kernel can be *faster* than fourth order interpolation, and sixth order interpolation has about the same efficiency as fifth order in terms of the overall number of mapped points. The "local work unit" kernel is sensitive to the way the PME mesh aligns to the neighbor list decomposition: for an interpolation order M, all atoms needed to complete the density throughout one of the neighbor list cells can be found in that cell and those directly adjacent so long as there are at least $(M-1)^3$ mesh points per neighbor list cell. For the calculations in Fig. 7, the combinations expected to produce forces of sufficient quality are fourth order interpolation and 5^3 mesh points per neighbor list cell, fifth order interpolation and 4^3 points per unit cell, or sixth order interpolation and 3^3 points per unit cell, with fifth order hitting the "sweet spot." In either of the other options, there are atoms in the outlying region that do not map anywhere in the work unit and a higher proportion of atoms with stencils that have a partial overlap with the work unit's relevant mapping region. Further optimizations in the unique "local work unit" scheme, which skip over some of these irrelevant atoms and stencil zones, may be possible, although they would also benefit the case of fifth order interpolation and 4^3 mesh points per unit cell. In addition, while a line determined by two points is not a trend, the RTX 4090 shows greater gains in the "local work units" mapping procedure than its FFTs relative to prior architectures. If future cards continue to deliver greater returns on `__shared__` memory `atomicAdd()` relative to FFTs, sixth order interpolation might become preferred.

05 September 2026 02:57:16

© Author(s) 2024

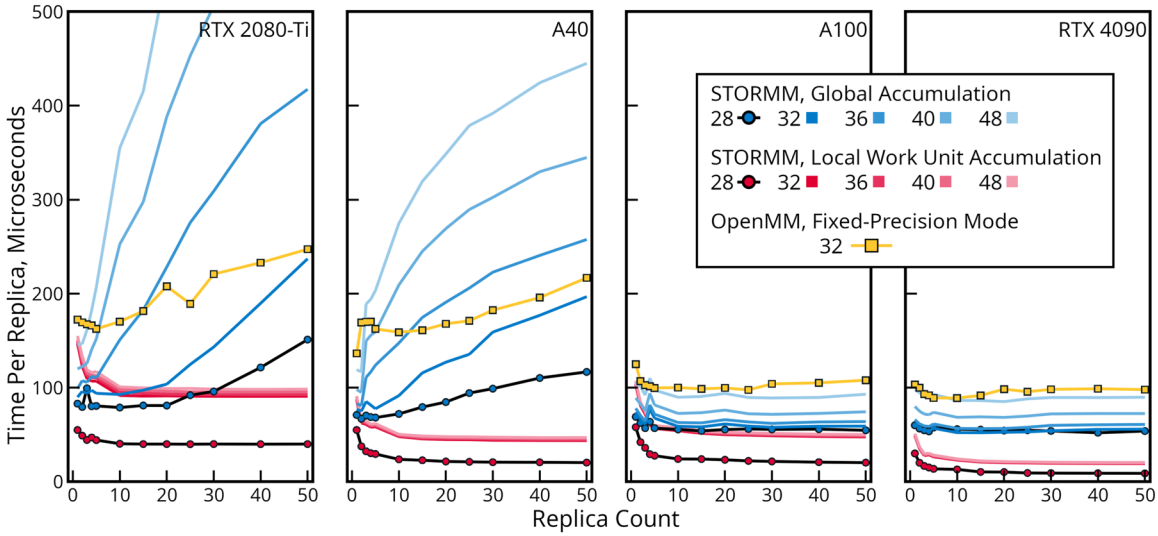


FIG. 9. Comparison of fixed-precision particle density accumulation in STORMM and OpenMM 8.1. The AMBER GPU code also uses fixed-precision accumulation, but with fourth order interpolation as opposed to the fifth order methods compared above. STORMM can operate in user-specified bitwise precision, whereas all facets of OpenMM's deterministic mode take 32 bits. STORMM calculations treat each replica of the 52 889 atom kinase simulation as distinct. For the purposes of comparison, OpenMM was tested with a tiled simulation box containing the equivalent number of replicas.

Figure 7 compares the most similar modes of each code, in terms of the level of numerical detail and overall sizes of the meshes. The density mapping kernels in STORMM are always designed to run in bitwise reproducible, fixed-precision mode, but they are tuned to detect the number of fixed precision bits, examine the topologies at hand, and (with consideration to the particle and mesh densities) determine whether it is safe to forego the secondary `int32_t` accumulator. For a typical protein-in-water topology like the kinase system examined in this section, STORMM can accumulate with up to 29 fixed-precision bits after the decimal without the secondary accumulator. This is similar to the level of detail captured in the AMBER code, although we took OpenMM's non-deterministic mode, accumulating in `float32_t`, as the closest match in terms of overall bandwidth requirements. Figure 9 shows the consequences of stepping into higher precision. Engaging the secondary accumulator to any degree, which STORMM does automatically for this system starting at 30 bits of fixed precision, requires waiting for and then processing of the return value from all atomic operations, and also that the secondary accumulators be stored somewhere: the "local work unit" kernel is designed to put them in `__shared__` memory at the cost of cutting the total mapped volume of any one work unit in half. Both factors are a significant hit to performance, often more than doubling the time taken to map particle density. Performance continues to degrade as the fixed precision model enters a regime where the secondary accumulator always accepts contributions. However, the performance remains very strong relative to the "global accumulation" kernel, which can crumble when faced with similar precision requirements, and the deterministic mapping procedure in OpenMM, which uses a 64-bit integer or a similar approach with paired `uint32_t` accumulators depending on the hardware. For their ability to handle multiple interpolation orders and also the user's choice of precision model, the STORMM density mapping kernels are perhaps the most versatile of any to date.

V. PRELIMINARY APPLICATIONS

The original methods in STORMM have applications to relevant problems, and the code base is a foundation for rapid development, but mature applications in conformer generation, docking, or molecular dynamics will take time to grow to a point where they compare to established programs and then go beyond the existing capabilities. While it is not yet possible to present mature applications, two features found in the alpha release of STORMM are detailed below. Both are built with the numerical methods detailed in Sec. IV A and IV B as well as data structures described throughout Sec. III. First, a large number of small molecules are subjected to combinatorial geometry permutations followed by energy minimization *en masse*. Next, dynamics of a series of fast-folding proteins will be explored with equilibrium molecular dynamics in implicit solvent. In both cases, the GPU is hundreds to thousands of times more powerful than a single-threaded, CPU-based program, and STORMM is many times more powerful than other GPU-enabled MD programs, if they are useful to problems of such scales at all.

A. Mass small molecule geometry optimization

The energy surfaces of molecules determine the conformations that they will adopt in solution, as well as how they will interact with protein targets. While it is not in the scope of this work to compare the current methods in STORMM with an established docking or

05 September 2026 02:57:16

© Author(s) 2024

conformer generation program, the utility of solvent models and minimization are matters of debate when selecting conformers.[38,39] Nonetheless, the relative cost of evaluating the solvation potential is much lower for a GPU than equivalent CPU code, due to an extreme increase in the relative cost of valence interactions (despite all the progress evident in Fig. 6). It is not until the number of particles P grows much larger that the $O(P^2)$ scaling of the non-bonded calculation dwarfs the valence calculations and global reductions in the net forces required by energy minimization, as is the case for simulations of small proteins discussed in Sec. V B and in Sec. 6 of the supplementary material.

Even without a strong interface to RDKit, STORMM is able to make intelligent decisions about the sampling spaces of small molecules by detecting their rotatable bonds (single bonds, not in ring systems) based on the input AMBER topologies. STORMM's conformer application accepts inputs for the number of rotamers N_{Rot} about each bond to attempt to sample. While this would lead to an explosive $(N_{\mathrm{Rot}})^K$ combinatorial possibilities for K rotatable bonds, the program can manage combinatorial sampling about individual bonds, pairs, or triplets of nearby rotatable bonds (randomizing other rotamer coordinates). It also accepts a hard upper limit to the number of rotamer combinations that should be attempted for any given molecule.

All the selected conformations can then be submitted to energy minimization according to the chosen force field parameters (encoded in AMBER topology format), plus the options of restraints and a choice of implicit solvent models as discussed in Secs. III E and II G. STORMM uses a clash criterion to determine whether a conformation is worth subjecting to energy minimization, and to keep forces within the bounds of the fixed-precision implementation, STORMM adds a "cooldown cycles" setting to the minimization namelist. As with AMBER, STORMM offers `ncyc` and `maxcyc` for the number of steepest descent and total line minimization cycles to attempt (the remainder performed by the conjugate gradient method), but `cdcyc` specifies the number of cycles over which to apply a softened potential, beginning with a user-specifiable degree of softening and dialing this back to zero (the natural potential is applied everywhere). There is no requirement that `cdcyc` be greater or smaller than `ncyc`. The electrostatic and van der Waals potentials are softened by stipulating an absolute distance or some proportion of the pairwise Lennard-Jones σ parameter at which the potential makes a transition to a quadratic or quartic function, continuous in the first derivative, such that the softened potential reaches a maximum value for distances $r = -1.0$ (this can never be reached, but defines a tractable form for the softened potential). In this way, most non-bonded interactions, perhaps all of them for any reasonable configuration, behave according to the natural potential, and conflicts are resolved as the applicable range of the softened potential recedes.

Results for 93 drug-like molecules (containing 3–6 rotatable bonds, some with chiral centers or isomerizable double bonds) are summarized in Table VI, which is intended to assess the practicality of STORMM's current features as a tool for automated generation of conformers and docked poses: the chemical model and details of the protocol will need revisions in order to produce competitive ensembles of compounds and candidates for any particular target. The FFEngine all-atom molecular mechanics force field was used to parameterize the ligands, along with different electrostatic effects. Ensembles of restraints were applied to prevent hydrogen bond donor and acceptor heavy atoms from coming closer than 2.8 Å to one another. With chiral centers or isomerizable double bonds and five or six rotatable bonds to sample, the coverage of the conformational space appears sparse, but the statistics reported in Table VI are taken after filtering severe clashes from the initial states. Furthermore, energy minimization guides the various conformers into many of the same local minima, producing about one "unique" conformer for every three initial states. While many conformer generation protocols are said to cause candidate poses to collapse into conformations irrelevant to the state of the molecule in solution or in a binding pocket, energy-minimization under all solvent models causes a moderate increase in the radius of gyration across the test set. Restraints preventing internal hydrogen bonding appear to have a minor effect, boosting this result by about 2% across all compounds and electrostatic models (data not shown). None

05 September 2026 02:57:16

TABLE VI. Summary statistics from conformational permutations and mass energy minimizations of 93 drug candidates. Compounds had at least three rotatable bonds, sampled by computing all permutations of up to three proximate bonds with random sampling of other degrees of freedom, including double bonds outside of ring systems and chiral centers. As of this writing, STORMM does not load libraries of non-aromatic ring conformations, which would be an additional source of permutations.

Potential[a]	Total sampled space (%)[b]	Unique results (%)[c]	Singularities[d] (%)	Strained conformations (%)[e]	ΔR_{gyr} (%)[f]	Best RMSD[g]
Generalized Born[h]	1.6×10^{-2}	32.4	0.2	0.1	13.1 ± 16.1	0.83 ± 0.36
Vacuum	1.6×10^{-2}	36.3	0.2	0.2	14.6 ± 18.3	0.86 ± 0.42
No charges	1.8×10^{-2}	32.6	0.0	0.1	12.4 ± 14.9	0.82 ± 0.38

[a]The proprietary FFEngine was used to model the compounds, with electrostatics as listed.
[b]Percentage (overall) of molecular conformations out of all possible permutations assembled by the CPU-based rotamer search, found to survive the severe clash check, and advance to energy minimization.
[c]Percentage of energy-minimized conformations distinguished by at least 0.5 Å heavy atom positional root mean-squared deviation (RMSD).
[d]Percentage of energy-minimized conformations registering exceptionally favorable energies, i.e., because a polar hydrogen had collapsed into a neighboring atom with negative charge.
[e]Percentage of energy-minimized conformations registering very high (+1000.0 kcal/mol) scores, indicating intractable geometries such as conjoined or "stabbed" rings.
[f]Average (percentage) change in the radius of gyration R_{gyr} as a consequence of energy minimization. Error bars are standard deviations collected for conformations in each compound.
[g]Minimum heavy-atom RMSD to the bound pose in each ligand's respective target.
[h]The modified "neck" generalized Born model.[40]

© Author(s) 2024

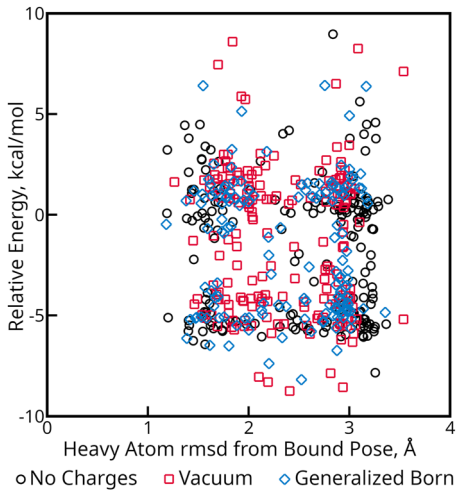


FIG. 10. Relative energies for optimized conformations of an example drug compound. The compound in question is taken from PDB 5BVE and has six rotatable bonds, as well as some isomerization centers that bifurcate the energy surface along two apparent "tiers" in the plot.

of the different methods show significant advantage in producing conformers closer to the bound state for each compound's respective target ligand.

The distances at which the softened non-bonded potentials engage may be specified in user input, but these trials underscored the importance of not softening the Lennard-Jones potential more than the electrostatic potential. Doing so can lead to singularities, in most cases involving polar hydrogens and their 1:4 neighbors or other atoms, which persist even after full restoration of the natural potential if the polar hydrogen has no Lennard-Jones character. Numerically, at the moment the forces or energies break the fixed-precision format, the minimizer can no longer make a decision on the direction of the move, leaving the system frozen in place as the ability to discern the difference between conformations at different points along the gradient line deteriorates more and more. To have the softened potentials begin at of 60% of the Lennard-Jones σ radii or 0.7 Å absolute distance between two charges appears to give reasonable results, but some configurations of some systems still fall into the trap. Restraints preventing internal hydrogen bonding were unable to mitigate a Lennard-Jones potential softened too far for its electrostatic counterpart (in some cases, the singularities involve polar hydrogens and 1:4 methyl carbons, which would not be protected by the restraints).

The spread of energies and conformations obtained from this exercise does not yield many insights into the best way to obtain conformations closest to each ligand's bound state, as illustrated in Fig. 10. The FFEngine all-atom molecular mechanics force field[41] used to parameterize the ligands puts conformations of disparate geometries at similar energy levels, and, given the strong anti-correlation of solvation energies with electrostatic internal energies, it may be preferable to omit both and solve conformations *in vacuo* with the van der Waals potential and bonded interactions alone.

B. Performance of implicit solvent molecular dynamics in STORMM

Before beginning performance evaluations of the code, a detailed validation of molecular dynamics in STORMM was undertaken as described in the supplementary material. Tests focused on energy conservation with a special emphasis on the effect of STORMM's unique constraint iteration protocol once it was discovered that, for most values of the time step and constraint

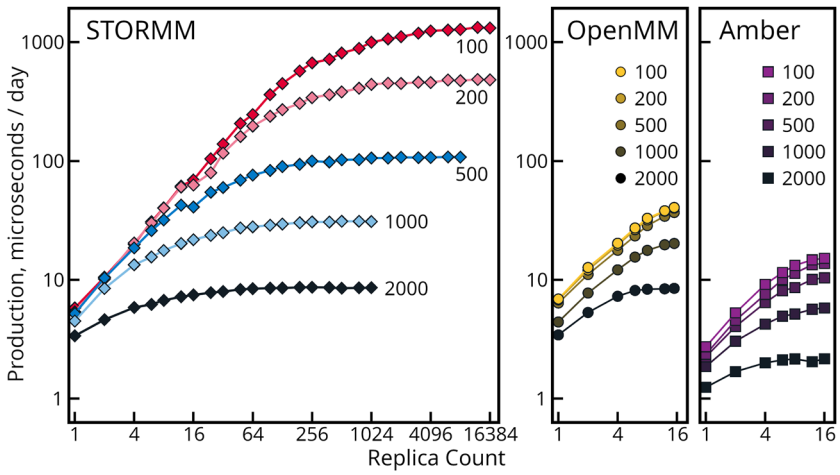


FIG. 11. Performance running multiple replicas of small implicit solvent simulations. The "neck" GB model[26] (`igb = 7` in AMBER, `GBn` in OpenMM) was used to model solvent effects. Peptide chains were taken from fragments of large proteins in the PDB (see Fig. 12 for systems of other sizes) and modeled with the ff14SB-"side chains only" force field. Hydrogen masses were repartitioned to three times their natural value. A 4 fs time step was used with a Langevin integrator (coupling constant 1.0/ps). System sizes are indicated on the plots or by inset legends. Plots compare the "production" modes of each code: STORMM with appropriate single-precision settings, single-precision OpenMM, and AMBER's SPFP mode. Results for RTX 4090 are presented: other cards' results differ in the overall scale of the throughput, which follows from the total compute capacity. STORMM calculations pertain to one runtime instance. AMBER and OpenMM calculations ran multiple concurrent executables using the NVIDIA MPS daemon.

05 September 2026 02:57:16

© Author(s) 2024

tolerance, STORMM was returning better energy conservation than Amber[42,43] and OpenMM[44] GPU codes. The STORMM iteration protocol was found to be on the whole more conservative. Although it requires more iterations to achieve a given convergence, STORMM's method distributes work better and can fulfill constraints for more complex groups built around a central atom, e.g., NH_4, without adding to the kernel register pressure.

The $O(N^2)$ complexity of typical generalized Born calculations, which either take no cutoffs or are of a size that most of their interactions fall within the cutoff anyway, implies poor scaling for small problems to the tens of thousands of threads on modern GPUs. STORMM's capability to stack multiple systems into a single runtime process obtains excellent GPU utilization with tens, hundreds, or even thousands of concurrent simulations, as shown in Fig. 11. Throughput may require many system replicas to "top out" on modern GPUs with a hundred thousand or more active threads in the major kernels' launch grids. Maximum throughput can be better understood in terms of the total non-bonded work in these generalized Born problems, and STORMM's non-bonded work units offer a seamless way for systems to flow together on the GPU. Each valence work unit described in Sec. IV B is exclusive to one system, but the non-bonded work units described in the supplementary material may incorporate tiles from multiple systems as the atom import space permits.

The card's memory is the hard limit on the number of independent systems that STORMM can run in parallel on a GPU. With NVIDIA's **M**ulti-**P**rocess **S**ervice (MPS), AMBER and OpenMM can also run multiple systems on a single card, although in separate runtime instances. MPS supports up to 16 concurrent processes (any more will begin to overload MPS lanes and conflict as would two concurrent jobs on the same GPU without the MPS daemon to interlace them). OpenMM throughput was found to peak at 15 jobs per GPU, whereas 16 AMBER jobs can run on one GPU through MPS. In the established codes, MPS is a means to get fivefold to

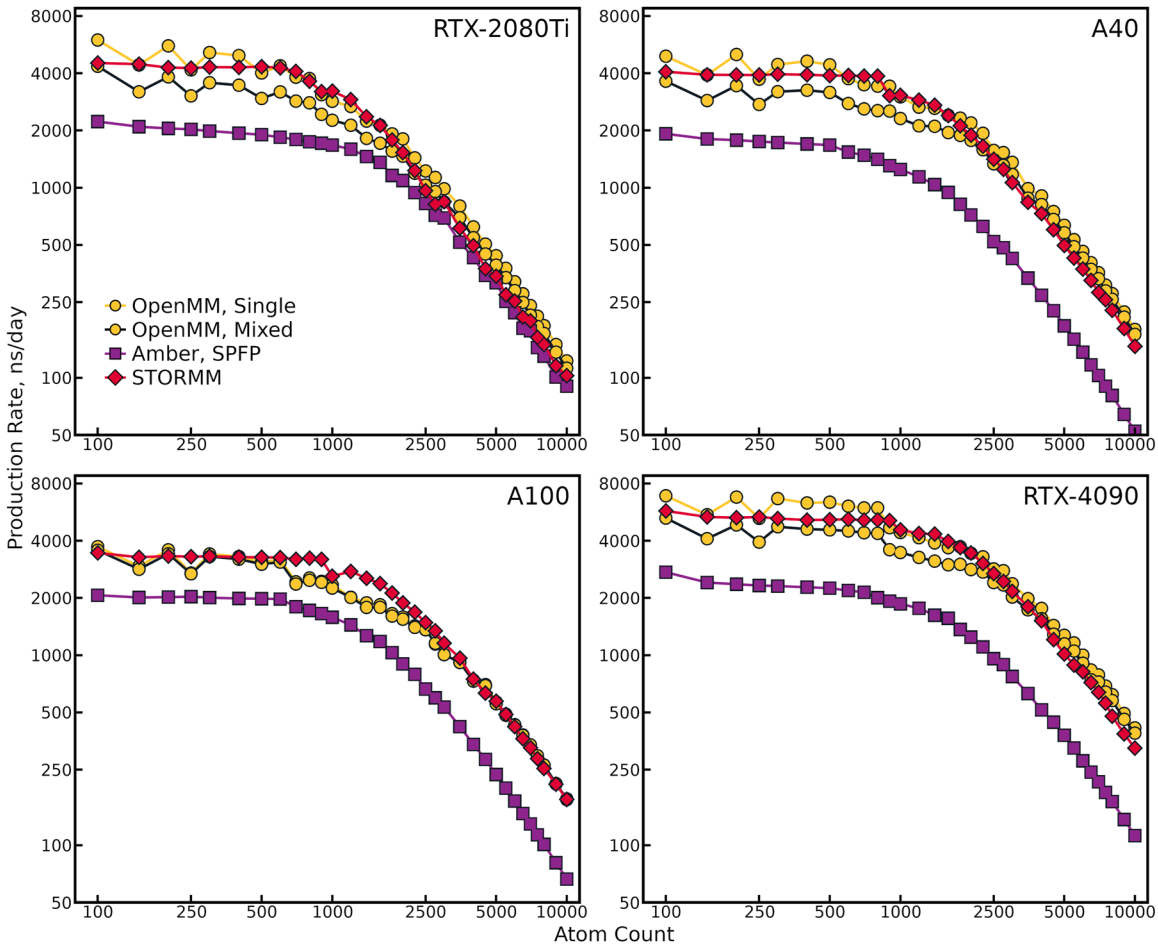


FIG. 12. Performance of implicit solvent simulations in three different codes. Simulations were guided by the AMBER ff14SB-"side chains only" force field and "neck" GB implicit solvent.[26] Hydrogen mass repartitioning (HMR) and a 4 fs time step were used with a Langevin thermostat (coupling constant 1.0/ps) in AMBER and STORMM and a `LangevinMiddleIntegrator` in OpenMM. The inset legend of the top left panel applies to all panels.

05 September 2026 02:57:16

© Author(s) 2024

sixfold throughput increases on small implicit solvent simulations. However, they do not keep up with the throughput available by running the same numbers of systems in a single STORMM runtime instance, or approach the throughput that STORMM can get on much larger collections of small problems. One additional advantage of STORMM, not shown in Fig. 11, is that the program can run separate topologies and build the work units to push all simulations forward at the same rate, so many systems of 200, 500, and 1000 atoms could all be submitted as part of the same batch and all would finish at the same time, with no extra effort to backfill a job queue running under MPS to ensure optimal utilization of the card.

The scaling on single simulations is also competitive, as shown in Fig. 12. In terms of numerics, STORMM's production mode falls somewhere between OpenMM's single-precision mode, applying constraints with `float32_t` arithmetic, and OpenMM's mixed-precision mode, storing its coordinates in 64-bit values (only when higher-precision settings are engaged does STORMM access the extra 32-bit accumulators in the `PhaseSpaceSynthesis`). Its performance follows suit for a range of common GB problem sizes, although at higher atom counts, OpenMM runs 20%–25% faster as it uses a more straightforward approach, whereas STORMM takes extra measures to make its non-bonded calculations more accurate and spread the calculations across more threads at low particle counts (see the supplementary material). AMBER has the poorest performance at small atom counts, but catches up on RTX-2080Ti as the system size grows to allow more thread engagement. On Ampere and later generation cards, however, AMBER's "neck" GB implementations[26,40] appear to deteriorate, scaling worse than $O(N^2)$ even as high atom counts should permit complete thread utilization in the cards.

Implicit solvent simulations of a few hundred particles could be a powerful tool for many applications in drug discovery, such as enhancing flexible docking calculations and predicting small molecule properties (e.g., solubility, aggregation, strain energy, and conformational ensembles). For general purpose molecular simulations, explicit water molecules and periodic boundary conditions are standard.

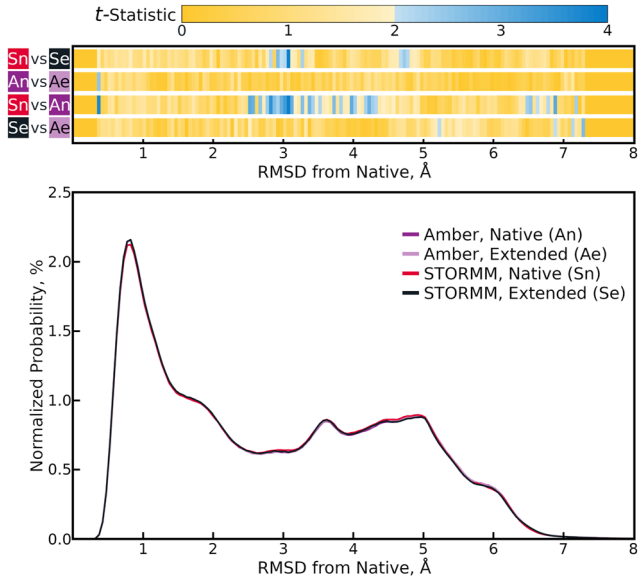


FIG. 13. C_α RMSD distributions for Trp-cage (tc5b) protein relative to the native state collected over four aggregated trajectories. Each pooled trajectory spans 2.4 ms. Histograms are normalized by the overall number of data points but not the estimated volume of states at a given RMSD (i.e., normalized by the squared value). The t-statistics obtained for comparisons between various pooled trajectories are shown in the upper panel, with the color scheme on the ordinate axis matching that in the inset legend of the lower panel. Thus, the lower two bands show the statistical significance of deviations between the abundance of structures with any given RMSD to the native state when comparing pooled trajectories generated by AMBER and by STORMM. The upper two bands provide context by showing the statistical significance of differences between pooled trajectories obtained with the same code but starting from different initial states (the "extended" or "native" tc5b conformations). To achieve 95% confidence that populations differ based on 32 independent measurements (separating any of the aggregated trajectories into 16 blocks and comparing block averages from two trajectories), the *t*-statistic must exceed 2.037. A difference between STORMM and AMBER results would have been declared had the lower two bands both showed significance (shades of blue in the heat map) at some RMSD value, while the upper two bands did not (yellow or gold in the heat map).

C. Comparing trajectories from STORMM and AMBER

To assess STORMM's molecular dynamics performance against an established code (AMBER22), the Trp-cage (tc5b) system was run using the modified "neck" GB solvent model, a 4.0 fs time step, and temperature regulation by a Langevin integrator with a collision frequency of 1.0 ps^{-1}. Hydrogen masses were repartitioned, and the constraint tolerance on their rigid bonds was set to 1.0×10^{-5}, following the protocol in a prior study.[40] Simulations were carried out for a total of 9.6 ms, hundreds of times longer than the original study, with either code running hundreds of separate trajectories started from either the native state (the first NMR structure in PDB 1L2Y) or an extended conformation. The first 1 μs of each trajectory was discarded to decorrelate the independent simulations. Results collected with both AMBER and STORMM are in quantitative agreement with those of the prior study, as shown in Fig. 13. The protein folds and unfolds many times during the simulations, and the highest populated conformation lies at about 0.8 Å root mean-squared deviation (RMSD) in the C_α positions of residues 3 through 18 relative to the native state. Secondary peaks at 3.6 and 5.0 Å RMSD are also seen, again in agreement with the earlier results. Simulations initiated from both the native state and the extended conformation appear to converge to the same profile in either code, and the distributions obtained by STORMM and AMBER show no differences reaching statistical significance after subdividing each set of trajectories into 16 parts and submitting the block averages to a two-tailed, unpaired Student's *t*-test ($p < 0.05$). In this comparison, a "significant difference" was defined as a patch of values in the RMSD histograms wherein STORMM and AMBER trajectories show different populations when started from both native and extended conformations, but neither code shows a significant difference between its own trajectories initiated from either starting point. To note, with only 300 μs of simulation time, the mean frequencies for each RMSD value appeared to diverge, thereby warranting the more extensive simulations and confidence testing described above, which indicated that the distributions are in fact the same.

05 September 2026 02:57:16

© Author(s) 2024

While the matching trajectory distributions and STORMM's energy conservation characteristics (see Sec. 6 of the supplementary material) are encouraging, questions linger from the analysis: does the energy drain created by a loose constraint tolerance and an aggressive time step, as is common practice in production simulations, have any observable effect on the outcome? If so, do STORMM's unique constraint iterations make any difference? To address these questions, the above protocol for tc5b was repeated once more, collecting another 4.8 ms of trajectory with STORMM for identical run conditions but taking a constraint tolerance of 1.0×10^{-7}. While the resources were not available to repeat the test with AMBER, the constraint solutions are expected to converge in the limit of a tight tolerance no matter the method of iteration. Block averaging over the 16 groups of equivalent simulations with either tolerance or starting configuration suggests that the equilibrium structural distributions are not influenced by the accuracy of the constraints: the distributions of C_α RMSDs show no differences of statistical significance, nor do χ_1 or χ_2 distributions for any of the residues differ in any way that approaches even 95% confidence. (Following from the comparison above, the experiment sought a modest arc of the rotamer range where different constraint tolerances produce different outcomes but alternate starting conditions do not, for $p < 0.05$. No residue of tc5b satisfied these criteria.) Furthermore, the autocorrelation time associated with each variable, a basic assessment of the dynamics, suggests that there are still no significant consequences to the choice of constraint tolerance. These results, in turn, suggest that STORMM's approach to SHAKE[45] and RATTLE[46] does not affect the simulation results.

The massive amount of simulation also provided a valuable "stress test" of the code base. One of the 640 STORMM trajectories taking a 1.0×10^{-5} tolerance did encounter some event, after about 1.6×10^9 steps, involving a very large velocity update from which the dynamics could not recover. This was investigated as part of the review process and traced to an oversight in the subtraction of split fixed-precision numbers (see Sec. IV A). The problem was studied by altering precision settings (e.g., numbers of bits for position, velocity, and force representations) such that 50 to 70 out of 1000 simulations would crash within 200 thousand steps (some of the crashes occurring even sooner), rather than waiting for one simulation out of two sets of 640 to crash after 1.6×10^9 steps. After guarding the subtractions, no simulations with the more sensitive settings were observed to crash even after much longer simulations. There were many places in the code where subtraction was performed on split fixed-precision numbers, and all have since been replaced with the guarded method. The guard adds a trivial amount of integer arithmetic to some processes, scaling as the number of bonded interactions in the system, and has no significant effect on run times. (Non-bonded interactions, including particle–mesh mapping, do not rely on split fixed-precision difference computations.)

Based on the precision models chosen throughout this study, it appears that an overwhelming majority of the problem manifested itself in the SHAKE constraints for lower-precision runs, where dual `int32_t` accumulators handled the evolving geometries. The cause of the crash is likely also at the root of the minor anomaly noted in Fig. S8 of the supplementary material. While more arithmetic errors must have occurred in the simulations presented in Fig. 13, the events remain rare, most would have been insignificant perturbations, and the bug is unlikely to have influenced the structural distributions. Knowing what the problem was and what settings would leave a simulation more vulnerable to corruption, it is clear that energy conservation in lower-precision STORMM simulations would have been affected before any other results. These simulations were repeated with the corrected code to confirm that the trends in Table S-T4 of the supplementary material were unaffected: STORMM's superior energy conservation is real. While a minor heating effect may have occurred due to the bug, it was several times smaller than the difference between STORMM's average energy drift and that of the other codes. The spread of STORMM's energy drifts in single-precision dynamics did diminish such that STORMM's energy drift rates became the most consistent of any of the engines' comparable run modes. Furthermore, the results in Fig. S9 of the supplementary material, finding equivalent results after comparing simulations with low-precision particle positions to simulations with high-precision coordinates that would be much less affected, corroborate the conclusion that the overall influence was negligible.

VI. BUILDING NEW PROGRAMS WITH STORMM

Current development priorities in STORMM are to expand and meld the existing capabilities with small molecule conformer generation, implicit solvent dynamics, and periodic molecular simulations for workflows in drug discovery. This effort will set up a continuum of methods between conformer generation and flexible docking with a central goal being a three-dimensional modeling and pose refinement system to balance the plethora of compounds created by generative modeling approaches[47,48] and then feed them into a state-of-the-art free energy calculation with all atoms of the protein, ligand, and solvent represented. Merging the fast screening methods with more demanding simulations in one programming framework will make drug discovery efforts more amenable to cloud computing resources. Implementing all the methods with the same underlying force fields (or deliberate choices to soften the potentials and alter the solvent model) may be a source of insights for computer-aided drug design.

To make conformer generation work in the context of a receptor is a form of flexible docking, whether a potential field that disfavors conformations, which would not fit in the binding site or a combination of that with some components of the binding site represented at the same atomic detail as the ligand itself. There are several improvements needed to bring STORMM's mass small molecule minimization to this degree of functionality, but they are already in progress. STORMM has a nascent capability for mapping receptor potentials and shapes to different three-dimensional meshes, the `BackgroundMesh` class, which can be found in the alpha code base available to the public. The class holds data that are similar to mesh generation in the particle mesh Ewald calculations described in Sec. IV C, but uses separate data structures and tricubic spline interpolation rather than *B*-spline mapping. With a detailed representation of the receptor's non-bonded potential and perhaps some mobile receptor side chains, solvation potentials and other details such as hydrogen bonding may become useful. The potential surface then becomes a third component of STORMM's complete energy function, in addition to the details of the system topology and sup-

05 September 2026 02:57:16

© Author(s) 2024

plemental restraints described in Sec. III. Guided by the gradients of that complete cost function, STORMM's multi-system energy minimization will enable high-throughput calculations within a flexible docking workflow.

Improvements to the minimization algorithm itself are also in order. The conjugate gradient approach described in Sec. V A is not efficient, especially because each line minimization move is carried out with three additional energy evaluations (the optimal step size is solved by finding the minimum of a cubic polynomial from points along the direction of travel). The Fast Inertial Relaxation Engine (FIRE) algorithm[49] takes a slight modification of the existing dynamics protocol and would optimize the structures in many fewer steps, which will become relevant as the number of energy minimizations scales with not just rotamer permutations but also poses within a binding site. While the potential softening approach was able to eliminate most of the conflicts that would make some initial guesses unusable, this aspect of the calculations can also be improved. The FIRE algorithm, or applying the RATTLE[46] algorithm to forces, would allow energy minimizations to proceed with rigid bond length constraints, which may broaden the range of acceptable onsets for the softened potentials. It may also be beneficial to borrow the approach of thermodynamic integration, including its "softcore" methods, whereby the electrostatic potential remains softened until the van der Waals potential is converged or close to its natural form.

STORMM's periodic dynamics will be able to stage millions of atoms on a single GPU, whether in a series of systems or even one very large simulation such as a virus capsid. Goals include a memory requirement on the order of 1500 bytes per atom, and the existing neighbor list object (as used in particle–mesh mapping in Sec. IV C) will accommodate up to 65 536 condensed-phase simulations in a single runtime instance. The design decision to align the particle–mesh interaction grid with the neighbor list decomposition carries profound consequences. In the early days of HPC molecular simulations, demand for pairwise interactions was expected to grow based on the perception of needing lower communication bandwidth, but pairs involve a lot of "misses" when not all particles in one tile interact. In contrast, the particle density mapping and FFTs are regular calculations where, in principle, GPUs excel. The STORMM particle–mesh setup requires that the spatial decomposition cells be cut very close to half the cutoff—if they become too large, the mesh density could fall and degrade the quality of the calculated forces. Pursuant to that need, a forthcoming paper will describe a fast method for taking the non-bonded migratory margin to zero and doing away with traditional GPU MD neighbor lists, instead updating the spatial decomposition at every step and checking the bonded exclusion status for every interacting pair. It is a departure from the way most codes frame the MD problem on GPUs and closer to the solution used in AceMD:[50] at any given time, a particle will be located inside a minimal parallelepiped, with no need to consider diffusion or a margin by which each decomposition cell should be padded. There is no effort to refresh a "phone book" of interacting tiles or exclusions: instead, it is all implied by the spatial decomposition and a static array of index-centered exclusion masks. As shown in Sec. IV C, this sets the stage for some surprising performance enhancements in the most memory-intensive facets of the workflow, and also straightforward methods for finding all particles in a particular vicinity for applications such as deep learning potential embedding[44,51] or QM/MM boundaries.[52,53]

Orthogonal to the dynamics engines, STORMM will gain new classes to manage groups of connected simulations, such as replica exchange,[54] weighted ensemble,[55] umbrella sampling,[56] or the nudged elastic band method.[57] Some of these features will make use of the `copyCoord` and `swapCoord` API to move coordinate sets, as well as the fact that coordinate objects have the option to be allocated exclusively in CPU or GPU memory for conserving resources. As demonstrated by the subtle bug noted in Secs. IV A and V C, it is also important to monitor simulations with automated tools to identify pathologies in the dynamics that an investigator running production simulations may not be looking for. The `Watcher` class is in development, which will scan for large forces, failed constraint convergence, and other details of the dynamics subject to the user's discretion. The minimal price of basic, constitutive tests will be "baked in" to future simulations, while more expensive checks can be requested as part of a *post mortem* if a calculation crashes.

Any of the classes in the STORMM libraries can be extended, with implicit GPU functionality, by elaborating one of the enumerator types to add new options or by adding additional `Hybrid` arrays containing the information of interest. For some extensions like polarizable molecular mechanics models, it may be better to repurpose some of the arrays already present in the `AtomGraph` object, such as atomic partial charges to be used in a fluctuating charge model.[58] New arrays would need to be added for quantities like multipole moments.[59] When incorporating new information, there are two major considerations: how will the new arrays interact with the class's existing functionality, and how will the new arrays affect the operation of other classes, in particular those with compressed data? When adding a new array, edits to custom copy or move constructors as well as abstracts may be needed but are trivial. As abstracts get larger, the price of passing them as arguments to GPU kernels increases incrementally, but if a class's sole abstract gets very large, it may be preferable to specialize different variants of it for one purpose or another. In a polarizable model that repurposes the existing `AtomGraph` and `AtomGraphSynthesis` partial charge arrays, it becomes important to consider how other classes in STORMM access partial charges. The neighbor list, as used in particle–mesh mapping (Sec. IV C), may store coordinates and an index for particle properties, including a charge parameter index. In a non-polarizable model, there may be a few hundred unique partial charge values, or a few thousand in a simulation with a bespoke protein model. In a polarizable model, every charge will have a unique value. The bit packing for STORMM's neighbor list details was, therefore, budgeted to give at least 23 bits to the charge index (more if the data are stored as a `float64_t` tuple), which would permit simulations of more than 8.3×10^6 polarizable atoms without making further alterations to the downstream `CellGrid` class.

More fluid organization within STORMM will also be supported by stronger integration with external molecular modeling environments through Python bindings. In the future, a strong interface between STORMM and RDKit[19] will improve STORMM's ability to interpret chemical information coming from user input, whether through additional file formats or instructions related to specific chemical groups. Such capabilities could also aid in identifying force field terms needed to develop molecular models in pursuit

05 September 2026 02:57:16

© Author(s) 2024

of high-throughput three-dimensional model rendering based on generative modeling in chemical space.[60] The modular design of molecular objects coupled with scalable algorithms in STORMM will support a range of simulation methods for managing many independent calculations with a common theme.

SUPPLEMENTARY MATERIAL

The supplementary material contains the following: benchmarks for Amber and OpenMM GPU codes running four common benchmarks; methodology and results for logarithmic adaptive tabulated splines in functions of one dimension; benchmarks for NVIDIA cuFFT library calls on problem sizes relevant to molecular dynamics; details of STORMM's constraints, non-bonded calculations, and random number generators; and validation of STORMM's implicit solvent dynamics with respect to energy conservation. All the supplementary material is arranged in a single manuscript of its own, with complete text and descriptions, designed to be as accessible as the main article itself. This information is available via the web at (link).

ACKNOWLEDGMENTS

The authors thank Peter Eastman, primary developer of OpenMM, for helpful conversations, as well as Peng Wang (NVIDIA Corporation) for technical expertise and advice. Ana Silveira (Psivant, LLC) contributed a set of drug candidates to test the conformer generator and helped in reviewing this article. Francesco Pontiggia (Harvard University, formerly Psivant, LLC) and Thomas Schultz (Psivant, LLC) assisted with logistical aspects of cluster resources and preparing the STORMM documentation. No government funding played a part in the development of STORMM.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

David S. Cerutti: Conceptualization (lead); Data curation (lead); Formal analysis (lead); Investigation (lead); Methodology (lead); Software (lead); Validation (lead); Writing – original draft (lead); Writing – review & editing (lead). **Rafal Wiewiora**: Data curation (supporting); Formal analysis (supporting); Investigation (supporting); Validation (supporting); Writing – review & editing (supporting). **Simon Boothroyd**: Conceptualization (supporting); Methodology (supporting); Software (supporting). **Woody Sherman**: Conceptualization (supporting); Formal analysis (supporting); Funding acquisition (lead); Project administration (lead); Resources (lead); Supervision (supporting); Writing – review & editing (supporting).

DATA AVAILABILITY

The STORMM code base enters an open-source, *alpha* state of testing with this publication. The libraries may be downloaded from the Psivant GitHub account at https://github.com/Psivant/stormm, including benchmarking programs for reproducing many of the results presented in this study. Interested parties may contact the corresponding author for help with additional tests and new code development based on STORMM's implementation.

REFERENCES

[1] Z. Cournia, B. Allen, and W. Sherman, "Relative binding free energy calculations in drug discovery: Recent advances and practical considerations," J. Chem. Inf. Model. **57**(12), 2911–2937 (2017).
[2] J. Harry Moore, C. Margreitter, J. Paul Janet, O. Engkvist, B. L. de Groot, and V. Gapsys, "Automated relative binding free energy calculations from SMILES to ΔΔG," Commun. Chem. **6**(1), 82 (2023).
[3] D. Ni, Y. Liu, R. Kong, Z. Yu, S. Lu, and J. Zhang, "Computational elucidation of allosteric communication in proteins for allosteric drug design," Drug Discovery Today **27**(8), 2226–2234 (2022).
[4] A. Chatzigoulas and Z. Cournia, "Rational design of allosteric modulators: Challenges and successes," Wiley Interdiscip. Rev.: Comput. Mol. Sci. **11**(6), e1529 (2021).
[5] A. Krämer, A. Ghysels, E. Wang, R. M. Venable, J. B. Klauda, B. R. Brooks, and R. W. Pastor, "Membrane permeability of small molecules from unbiased molecular dynamics simulations," J. Chem. Phys. **153**, 124107 (2020).
[6] S. Hossain, A. Kabedev, A. Parrow, C. A. S. Bergström, and P. Larsson, "Molecular simulation as a computational pharmaceutics tool to predict drug solubility, solubilization processes and partitioning," Eur. J. Pharm. Biopharm. **137**, 46–55 (2019).
[7] C. Yang, G. Tong, T. Wang, R. Patel, Q. Xiong, S. Ahmed, C. Wu, J. Sheng, C. Lin, V. Sachdeva, W. Sherman, and M. Herbordt, "Fully integrated FPGA molecular dynamics simulations," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Computing Machinery, New York, NY USA, 2019), pp. 1–31.
[8] M. J. Harvey, and G. De Fabritiis, "An Implementation of the Smooth Particle Mesh Ewald Method on GPU Hardware," J. Chem. Theory Comput. **5**(9), 2371–2377 (2009).
[9] K. J. Bowers, E. Chow, H. Xu, R. O. Dror, M. P. Eastwood, B. A. Gregersen, J. L. Klepeis, I. Kolossvary, M. A. Moraes, F. D. Sacerdoti, J. K. Salmon, Y. S. Shan, and D. E. Shaw, "Scalable algorithms for molecular dynamics simulations on commodity clusters," in *Proceedings of the 2006 ACM/IEEE Conference on Supercomputing* (IEEE, 2006), p. 84.
[10] J. A. Anderson, J. Glaser, and S. C. Glotzer, "HOOMD-blue: A Python package for high-performance molecular dynamics and hard particle Monte Carlo simulations," Comput. Mater. Sci. **173**, 109363 (2020).
[11] M. James Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, and E. Lindahl, "GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers," SoftwareX **1–2**, 19–25 (2015).
[12] Á. Jàsz, Á. Rák, I. Ladjánszki, and G. Cserey, "Classical molecular dynamics on graphics processing unit architectures," Wiley Interdiscip. Rev.: Comput. Mol. Sci. **10**(2), e1444 (2020).
[13] J. Ding, S. Tang, Z. Mei, L. Wang, Q. Huang, H. Hu, M. Ling, and J. Wu, "Vina-GPU 2.0: Further accelerating autodock Vina and its derivatives with graphics processing units," J. Chem. Inf. Model. **63**(7), 1982–1998 (2023).
[14] S. Seritan, C. Bannwarth, B. S. Fales, E. G. Hohenstein, C. M. Isborn, S. I. L. Kokkila-Schumacher, X. Li, F. Liu, N. Luehr, J. W. Snyder, Jr., C. Song, A. V. Titov, I. S. Ufimtsev, L.-P. Wang, and T. J. Martínez, "TeraChem: A graphical processing unit-accelerated electronic structure package for large-scale ab initio molecular dynamics," Wiley Interdiscip. Rev.: Comput. Mol. Sci. **11**(2), e1494 (2021).
[15] N. Nethercote and J. Seward, "How to shadow every byte of memory used by a program," in *VEE'07: Proceedings of the 3rd International Conference on Virtual Execution Environments* (Association for Computing Machinery, New York, NY, 2007).
[16] J. W. Eaton, D. Bateman, S. Hauberg, and R. Wehbring, GNU Octave version 6.3.0 manual: A high-level interactive language for numerical computations, 2021.

05 September 2026 02:57:16

© Author(s) 2024

[17]C. R. Harris, K. Jarrod Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. Fernández del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," Nature **585**(7825), 357–362 (2020).
[18]J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng. **9**(3), 90–95 (2007).
[19]See https://www.rdkit.org for RDKit Contributors, RDKit: Open-source cheminformatics.
[20]A. Katharina Dehof, A. Rurainski, Q. B. A. Bui, S. Böcker, H.-P. Lenhof, and A. Hildebrandt, "Automated bond order assignment as an optimization problem," Bioinformatics **27**(5), 619–625 (2011).
[21]J. Wang, W. Wang, P. A. Kollman, and D. A. Case, "Automatic atom type and bond type perception in molecular mechanical calculations," J. Mol. Graphics Modell. **25**(2), 247–260 (2006).
[22]I. D. Welsh and J. R. Allison, "Automated simultaneous assignment of bond orders and formal charges," J. Cheminf. **11**, 18 (2019).
[23]W. Zeng and R. L. Church, "Finding shortest paths on real road networks: The case for A*," Int. J. Geogr. Inf. Sci. **23**(4), 531–543 (2009).
[24]M. Buck, S. Bouguet-Bonnet, R. W. Pastor, and A. D. MacKerell, "Importance of the CMAP correction to the CHARMM22 protein force field: Dynamics of hen lysozyme," Biophys. J. **90**(4), L36–L38 (2006).
[25]E. Ulrich, L. Perera, M. L. Berkowitz, T. Darden, H. Lee, and L. G. Pedersen, "A smooth particle mesh Ewald method," J. Chem. Phys. **103**, 8577–8593 (1995).
[26]J. Mongan, C. Simmerling, J. Andrew McCammon, D. A. Case, and A. Onufriev, "Generalized Born model with a simple, robust molecular volume correction," J. Chem. Theory Comput. **3**(1), 156–169 (2007).
[27]A. Onufriev, D. Bashford, and D. A. Case, "Exploring protein native states and large-scale conformational changes with a modified generalized born model," Proteins: Struct., Funct., Bioinf. **55**(2), 383–394 (2004).
[28]G. D. Hawkins, C. J. Crame, and D. G. Truhlar, "Pairwise solute descreening of solute charges from a dielectric medium," Chem. Phys. Lett. **246**(1–2), 122–129 (1995).
[29]D. A. Case, T. E. Cheatham III, T. Darden, H. Gohlke, R. Luo, K. M. Merz, Jr., A. Onufriev, C. Simmerling, B. Wang, and R. J. Woods, "The Amber biomolecular simulation programs," J. Comput. Chem. **26**, 1668 (2005).
[30]B. R. Brooks, C. L. Brooks III, A. D. Mackerell, Jr., L. Nilsson, R. J. Petrella, B. Roux, Y. Won, G. Archontis, C. Bartels, S. Boresch, A. Caflisch, L. Caves, Q. Cui, A. R. Dinner, M. Feig, S. Fischer, J. Gao, M. Hodoscek, W. Im, K. Kuczera, T. Lazaridis, J. Ma, V. Ovchinnikov, E. Paci, R. W. Pastor, C. B. Post, J. Z. Pu, M. Schaefer, B. Tidor, R. M. Venable, H. L. Woodcock, X. Wu, W. Yang, D. M. York, and M. Karplus, "CHARMM: The biomolecular simulation program," J. Comput. Chem. **30**(10), 1545–1614 (2009).
[31]T.-S. Lee, D. S. Cerutti, D. Mermelstein, C. Lin, S. LeGrand, T. J. Giese, A. Roitberg, D. A. Case, R. C. Walker, and D. M. York, "GPU-accelerated molecular dynamics and free energy methods in Amber18: Performance enhancements and new features," J. Chem. Inf. Model. **58**(10), 2043–2050 (2018).
[32]P. Eastman, J. Swails, J. D. Chodera, R. T. McGibbon, Y. Zhao, K. A. Beauchamp, L.-P. Wang, A. C. Simmonett, M. P. Harrigan, C. D. Stern, R. P. Wiewiora, B. R. Brooks, and V. S. Pande, "OpenMM 7: Rapid development of high performance algorithms for molecular dynamics," PLoS Comput. Biol. **13**(7), e1005659 (2017).
[33]R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles* (Taylor & Francis, 1988).
[34]D. J. Hardy, Z. Wu, J. C. Phillips, J. E. Stone, R. D. Skeel, and K. Schulten, "Multilevel summation method for electrostatic force evaluation," J. Chem. Theory Comput. **11**(2), 766–779 (2015).
[35]W. Michael Brown, A. Kohlmeyer, S. J. Plimpton, and A. N. Tharrington, "Implementing molecular dynamics on hybrid high performance computers–particle–particle particle-mesh," Comput. Phys. Commun. **183**(3), 449–459 (2012).
[36]D. S. Cerutti and D. A. Case, "Multi-level Ewald: A hybrid multigrid/fast Fourier transform approach to the electrostatic particle-mesh problem," J. Chem. Theory Comput. **6**(2), 443–458 (2010).
[37]D. S. Cerutti, R. E. Duke, T. A. Darden, and T. P. Lybrand, "Staggered mesh Ewald: An extension of the smooth particle-mesh Ewald method adding great versatility," J. Chem. Theory Comput. **5**(9), 2322–2338 (2009).
[38]A. T. McNutt, F. Bisiriyu, S. Song, A. Vyas, G. R. Hutchison, and D. Ryan Koes, "Conformer generation for structure-based drug design: How many and how good?," J. Chem. Inf. Model. **63**(21), 6598–6607 (2023).
[39]P. C. D. Hawkins, A. Geoffrey Skillman, G. L. Warren, B. A. Ellingson, and M. T. Stahl, "Conformer generation with OMEGA: Algorithm and validation using high quality structures from the protein databank and cambridge structural database," J. Chem. Inf. Model. **50**(4), 572–584 (2010).
[40]H. Nguyen, D. R. Roe, and C. Simmerling, "Improved generalized Born solvent model parameters for protein simulations," J. Chem. Theory Comput. **9**(4), 2020–2034 (2013).
[41]S. Khuttan, S. Azimi, J. Z. Wu, S. Dick, C. Wu, H. Xu, and E. Gallichio, "Taming multiple binding poses in alchemical binding free energy prediction: The β-cyclodextrin host–guest SAMPL9 blinded challenge," Phys. Chem. Chem. Phys. **25**, 24364–24376 (2023).
[42]R. Salomon-Ferrer, A. W. Götz, D. Poole, S. Le Grand, and R. C. Walker, "Routine microsecond molecular dynamics simulations with AMBER on GPUs. 2. Explicit solvent particle mesh Ewald," J. Chem. Theory Comput. **9**(9), 3878–3888 (2013).
[43]A. W. Götz, M. J. Williamson, D. Xu, D. Poole, S. Le Grand, and R. C. Walker, "Routine microsecond molecular dynamics simulations with AMBER on GPUs. 1. Generalized Born," J. Chem. Theory Comput. **8**(5), 1542–1555 (2012).
[44]P. Eastman, R. Galvelis, R. P. Peláez, C. R. A. Abreu, S. E. Farr, E. Gallicchio, A. Gorenko, M. M. Henry, F. Hu, J. Huang, A. Krämer, J. Michel, J. A. Mitchell, V. S. Pande, J. P. Rodrigues, J. Rodriguez-Guerra, A. C. Simmonett, S. Singh, J. Swails, P. Turner, Y. Wang, I. Zhang, J. D. Chodera, G. De Fabritiis, and T. E. Markland, "OpenMM 8: Molecular dynamics simulation with machine learning potentials," J. Phys. Chem. B **128**(1), 109–116 (2024).
[45]J.-P. Ryckaert, G. Ciccotti, and H. J. C. Berendsen, "Numerical integration of the Cartesian equations of motion of a system with constraints: Molecular dynamics of *n*-alkanes," J. Comput. Phys. **23**(3), 327–341 (1977).
[46]H. C. Andersen, "Rattle: A 'velocity' version of the shake algorithm for molecular dynamics calculations," J. Comput. Phys. **52**(1), 24–34 (1983).
[47]V. Fialková, J. Zhao, K. Papadopoulos, O. Engkvist, E. Jannik Bjerrum, T. Kogej, and A. Patronov, "LibINVENT: Reaction-based generative scaffold decoration for in silico library design," J. Chem. Inf. Model. **62**(9), 2046–2063 (2022).
[48]J. B. Ingraham, M. Baranov, Z. Costello, K. W. Barber, W. Wang, I. Ahmed, F. Vincent, D. M. Lord, C. Ng-Thow-Hing, E. R. Van Vlack, S. Tie, X. Vincent, S. C. Cowles, A. Leung, J. V. Rodrigues, C. L. Morales-Perez, A. M. Ayoub, R. Green, K. Puentes, O. Frank, N. V. Panwar, O. Fritz, A. R. Root, A. L. Beam, F. J. Poelwijk, and G. Grigoryan, "Illuminating protein space with a programmable generative model," Nature **623**, 1070–1078 (2023).
[49]E. Bitzek, P. Koskinen, F. Gähler, M. Moseler, and P. Gumbsch, "Structural relaxation made simple," Phys. Rev. Lett. **97**, 170201 (2006).
[50]M. J. Harvey, G. Giupponi, and G. D. Fabritiis, "ACEMD: Accelerating biomolecular dynamics in the microsecond time scale," J. Chem. Theory Comput. **5**(6), 1632–1639 (2009).
[51]S. Doerr, M. Majewski, A. Pérez, A. Krämer, C. Clementi, F. Noe, T. Giorgino, and G. D. Fabritiis, "TorchMD: A deep learning framework for molecular simulations," J. Chem. Theory Comput. **17**(4), 2355–2363 (2021).
[52]C. E. Tzeliou, M. A. Mermigki, and D. Tzeli, "Review on the QM/MM methodologies and their application to metalloproteins," Molecules **27**(9), 2660 (2022).
[53]Q. Cui, T. Pal, and L. Xie, "Biomolecular QM/MM simulations: What are some of the 'burning issues,'" J. Phys. Chem. B **125**(3), 689–702 (2021).
[54]R. C. Bernardi, M. C. R. Melo, and K. Schulten, "Enhanced sampling techniques in molecular dynamics simulations of biological systems," Biochim. Biophys. Acta, Gen. Subj. **1850**(5), 872–877 (2015).
[55]D. M. Zuckerman and L. T. Chong, "Weighted ensemble simulation: Review of methodology, applications, and software," Annu. Rev. Biophys. **46**, 43–57 (2017).

05 September 2026 02:57:16

© Author(s) 2024

[56]W. You, Z. Tang, and C. A. Chang, "Potential mean force from umbrella sampling simulations: What can we learn and what is missed?," J. Chem. Theory Comput. **15**(4), 2433–2443 (2019).
[57]E. L. Kolsbjerg, M. N. Groves, and B. Hammer, "An automated nudged elastic band method," J. Chem. Phys. **145**(9), 094107 (2016).
[58]J. Chen and T. J. Martínez, "QTPIE: Charge transfer with polarization current equalization. A fluctuating charge model with correct asymptotics," Chem. Phys. Lett. **438**(4–6), 315–320 (2007).
[59]J. W. Ponder, C. Wu, P. Ren, V. S. Pande, J. D. Chodera, M. J. Schnieders, I. Haque, D. L. Mobley, D. S. Lambrecht, R. A. DiStasio, Jr., M. Head-Gordon, G. N. I. Clark, M. E. Johnson, and T. Head-Gordon, "Current status of the AMOEBA polarizable force field," J. Phys. Chem. B **114**(8), 2549–2564 (2010).
[60]D. M. Anstine and O. Isayev, "Generative models as an emerging paradigm in the chemical sciences," J. Am. Chem. Soc. **145**(16), 8736–8750 (2023).

05 September 2026 02:57:16

© Author(s) 2024